

StimGen 2

**A compact, symbolic language for electrophysiological stimulus waveforms:
concepts, semantics, and reference specification**

Michele Giugliano

Draft — October 2026

We describe StimGen 2, a symbolic meta-description for the synthesis of digital stimulus waveforms in electrophysiology. A waveform is written as a short text, independent of its length, sampling rate, and D/A resolution, and is converted into a sequence of samples only at the moment of use. We give the language a precise mathematical meaning. We separate three layers (waveform, multichannel stimulus, and protocol). We define the sampling of a continuous-time description, the algebra by which waveforms are combined, and the statistical semantics of stochastic components together with their reproducibility. Descriptions are stored as plain text files with the extension `.sg`.

Contents

I. Tutorial	7
1. Introduction	8
1.1. Why describe stimuli as text	8
1.2. Three layers	9
1.3. How to read this document	9
1.4. Acknowledgements	9
2. A first stimulus	10
2.1. A current step	10
2.1.1. Durations and units	10
2.1.2. Negative values	11
2.2. Generators and their parameters	11
2.3. Noise	12
2.4. Pulse trains	13
2.5. From description to samples	13
3. Combining and transforming waveforms	14
3.1. Arithmetic on generators	14
3.2. Transformations	14
3.3. Ramps that continue	15
3.4. Blocks: waveforms inside waveforms	15
3.5. Repeating a part of a waveform	16
3.6. A larger example	16
4. Multichannel stimuli and markers	18
4.1. Two outputs	18
4.2. Markers	18
4.3. Digital outputs	19
4.4. Same file, same noise?	19
5. Protocols: repetitions and sweeps	21
5.1. A family of current steps	21
5.2. Several variables	21
5.3. Frozen noise and reliability	22
5.4. Protocols and directories	23
II. Reference specification	24
6. Conventions	25
6.1. Requirement levels	25
6.2. Mathematical notation	25
6.3. Units	26
6.3.1. Base units	26

6.3.2. Amplitudes	26
6.4. Numbers	27
6.5. Files	27
6.6. Conformance	27
7. The abstract model	28
7.1. Three layers	28
7.2. Generators	28
7.3. Timed waveforms	29
7.4. Concatenation	29
7.5. Pointwise combination	30
7.6. Context: the previous value	30
7.7. Meaning of a description	31
7.8. Summary of the objects	31
8. Sampling and quantisation	32
8.1. The sampling grid	32
8.2. Segment boundaries	32
8.3. Local time of samples	33
8.4. End value of a segment	34
8.5. Global clock	34
8.6. Values outside the waveform	34
8.7. Aliasing and edges	35
8.8. Quantisation	35
9. Catalogue of primitive generators	36
9.1. Levels and ramps	36
9.1.1. <code>dc</code> : constant level	36
9.1.2. <code>ramp</code> : linear ramp	38
9.1.3. <code>relax</code> : exponential relaxation	38
9.2. Periodic generators	38
9.2.1. <code>sine</code> : sinusoid	39
9.2.2. <code>square</code> : square wave	39
9.2.3. <code>saw</code> : sawtooth	39
9.2.4. <code>triangle</code> : symmetric triangle	40
9.2.5. <code>chirp</code> : swept-frequency sinusoid	40
9.3. Synaptic-like waveforms	40
9.3.1. <code>biexp</code> : difference of exponentials	40
9.3.2. <code>alpha</code> : alpha function	41
9.4. Arbitrary samples	41
9.4.1. <code>file</code> : samples from a file	41
9.5. Pulse trains	42
9.5.1. <code>pulses</code> : train of pulses	42
9.6. Noise processes	44
9.6.1. <code>ou</code> : Ornstein–Uhlenbeck process	44
9.6.2. <code>wnoise</code> : Gaussian white noise	44
9.6.3. <code>unoise</code> : uniform white noise	45
9.6.4. <code>cnoise</code> : power-law noise	45
10. Algebra of waveforms	46
10.1. Expressions	46
10.1.1. Precedence	46

10.2. Duration rules	47
10.2.1. Nested sequences and sampling	47
10.3. Binary operators	47
10.4. Unary maps	47
10.5. Repetition inside a waveform	48
10.6. The previous value	48
10.6.1. End values of expressions	48
10.6.2. Resolution of <code>prev</code>	49
10.7. Static checks	49
10.8. Algebraic laws (informative)	49
10.9. Examples	50
11. Stochastic semantics and reproducibility	51
11.1. Process and realisation	51
11.2. Random streams	51
11.3. Keys and addresses	52
11.3.1. Addresses	52
11.3.2. Keys	52
11.3.3. The master seed	53
11.4. From words to variates	53
11.5. Reproducibility levels	53
11.6. Repetitions and trials	54
12. The stimulus layer	55
12.1. Definition	55
12.2. Channels	55
12.2.1. Units and dimensional checks	56
12.2.2. Copies of a channel	56
12.2.3. Digital channels	56
12.3. Duration and alignment	56
12.4. Markers	57
12.5. Sampling rate and seed	57
12.6. A waveform is also a stimulus	57
13. The protocol layer	58
13.1. A first protocol	58
13.2. Elements of a protocol	58
13.2.1. Variables and substitution	59
13.2.2. Sweep values	59
13.2.3. Conditions	59
13.3. Expansion semantics	60
13.3.1. Ordering	60
13.3.2. Timing between trials	60
13.3.3. Seeds of the trials	61
13.4. The directory form	61
14. Concrete syntax	63
14.1. Lexical rules	63
14.2. Grammar	64
14.2.1. Files	64
14.2.2. Waveforms	64
14.2.3. Stimuli	65

14.2.4. Protocols	65
14.2.5. Index files	65
14.3. Positional and named arguments	65
14.4. Canonical form	66
14.5. Command line	66
15. Output and provenance	67
15.1. Realised sample files	67
15.2. The provenance record	68
15.3. Errors and warnings	69
A. Glossary	70
B. Mathematical derivations	72
B.1. Error of segment boundaries	72
B.2. Exact update of the Ornstein–Uhlenbeck process	72
B.3. Peak and normalisation of the biexponential	73
B.4. Phase of the chirp	73
B.5. Spectrum of discrete white noise	73
B.6. Synthesis of power-law noise	73
References	75

Part I.

Tutorial

1. Introduction

In a typical intracellular experiment, a neuron receives a sequence of current or voltage commands: a family of steps to measure its input resistance and firing rate, a ramp to find its rheobase, a chirp to measure its impedance, a noisy current to probe the reliability of its spike times. Each of these commands is, in the end, a list of numbers sent to a D/A converter at a fixed rate. Yet nobody thinks of a stimulus as a list of numbers. We think of it as “half a second at rest, then one second at 300 pA, then back to rest”.

StimGen 2 is a small language that lets us write stimuli exactly as we think of them. The step above is the text

```
500ms  dc(0)
1s      dc(300)
500ms  dc(0)
```

and a five-second sinusoid riding on a noisy background is

```
5s  sine(amp=50, freq=8Hz) + ou(mean=0, sd=20, tau=5ms)
```

These texts are converted into samples only when the stimulus is played.

1.1. Why describe stimuli as text

Writing stimuli as short symbolic texts, rather than storing their samples, has several advantages, and they are the design goals of StimGen 2.

- **Compactness.** The length of a description does not grow with the duration of the stimulus. Ten minutes of noise take one line.
- **Independence of the hardware.** A description does not depend on the sampling rate or on the resolution of the converter. The same file can be played at 10 kHz on one set-up and at 50 kHz on another.
- **Readability and editability.** A stimulus can be read, checked, and modified in any text editor, generated by a script, or typed directly on the command line.
- **Precision.** Every description has one exact meaning, defined mathematically in Part II, so that two implementations produce the same samples.
- **Reproducibility.** Stochastic stimuli are reproducible from their description and a seed, and the seed is always recorded. Every stimulus that was ever played can be regenerated, years later, from the record stored with the data.

1.2. Three layers

StimGen 2 distinguishes three layers, each built on the previous one.

1. A **waveform** describes what one output does during one trial. It is a sequence of *segments*, each lasting a given time and built from elementary *generators* such as constants, ramps, sinusoids, and noise processes. Generators can be added, multiplied, and transformed.
2. A **stimulus** combines several waveforms on named output channels, with their units and a common time base, and adds *markers* that label instants of interest in the recorded data.
3. A **protocol** describes a whole set of trials: which parameter is varied and over which values, how many times each condition is repeated, in which order, and how much time separates the trials.

Most users will spend most of their time with waveforms. Stimuli and protocols are needed as soon as an experiment involves more than one output or more than one trial, which is to say very soon.

All three are written in plain text files with the extension `.sg`.

1.3. How to read this document

Part I, which you are reading, is a tutorial. It introduces the language through examples and assumes no more than familiarity with electrophysiology. Part II is the reference specification: it defines every construct precisely, using a little mathematics, and is written for implementers and for users who want to know exactly what a description does. The tutorial refers to the relevant sections of Part II as it goes. Appendix A is a glossary, and Appendix B contains the derivations of the formulas.

Every figure in this document was produced by the reference renderer `sg` from the StimGen 2 text printed next to it, and plotted with the small reader `tools/sgplot.py` (`docs/figures/make_figures.py` regenerates them all).

1.4. Acknowledgements

The idea of describing stimulus waveforms by short parametric texts, and of running electrophysiology experiments from the command line, grew over many years of work, first with Maura Arsiero at the University of Bern, and later with Daniele Linaro and João Couto ([Linaro et al. 2014](#)). StimGen 2 is a new design, but it owes its spirit to that work.

2. A first stimulus

This chapter introduces waveforms: segments, durations, generators, and units. All examples are complete `.sg` files.

2.1. A current step

The simplest stimulus is a step:

```
# step.sg: a 1 s depolarising step
500ms  dc(0)
1s      dc(300)
500ms  dc(0)
```

Each line is a *segment*. It starts with a duration and continues with what the output does during that time. `dc(level)` holds a constant level. The segments are played one after the other, so this waveform lasts 2 s. Everything after a `#` is a comment. Fig. 2.1 shows the result.

The waveform does not say whether 300 means picoamperes or millivolts. That is decided by the channel it is played on (Sec. 4). We may still write the unit, as in `dc(300pA)`, and StimGen 2 will check that it matches the channel.

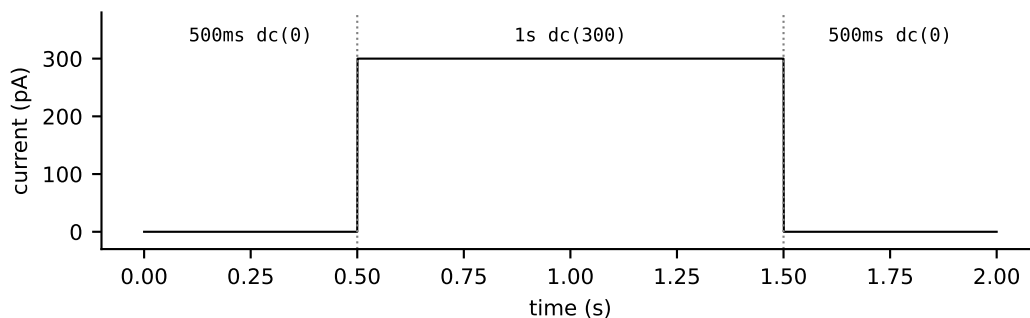


Figure 2.1.: The step waveform of `step.sg`, rendered by `sg` at 20 kHz. Dotted lines mark the boundaries of the three segments.

2.1.1. Durations and units

Durations are written with a unit: `500ms`, `1s`, `2min`, `250us`. The unit of a segment duration is required, so `1 dc(0)` is an error. For other parameters a number without a unit is in the canonical unit (seconds, hertz, radians), but we recommend always writing the unit: frequencies in `Hz` or `kHz`, phases in `rad` or `deg`, and times in `s`, `ms`, or `us`. Writing the wrong kind of unit, such as a frequency of `10ms`, is an error, not a guess (Sec. 6.3).

2.1.2. Negative values

Negative numbers need no special care:

```
500ms  dc(0)
1s      dc(-100)
500ms  dc(0)
```

2.2. Generators and their parameters

`dc` is one of sixteen *generators* listed in Sec. 9. A generator takes parameters, given by position or by name:

```
5s  sine(50, 8Hz)                # by position
5s  sine(amp=50, freq=8Hz)        # by name, same thing
5s  sine(50, 8Hz, phase=90deg, offset=10)
```

Parameters that are not given take their default values, listed in the card of each generator. For example, `phase` and `offset` of `sine` default to zero. Naming parameters makes a file easier to read later, and we use names in most examples.

A few generators of everyday use:

```
2s    ramp(from=0, to=500)        # linear ramp
5s     square(amp=100, freq=2Hz, duty=25%)
10s    chirp(amp=50, f0=0.5Hz, f1=20Hz) # frequency sweep (ZAP)
200ms  biexp(amp=30, tau_rise=1ms, tau_decay=8ms) # synaptic-like current
```

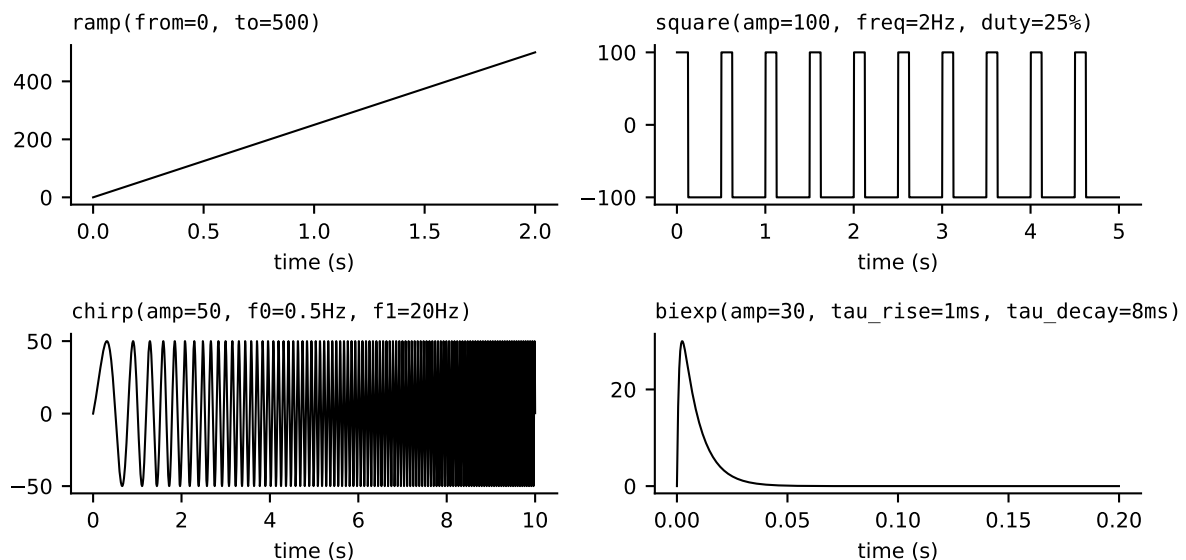


Figure 2.2.: The four generators of the example above, each rendered as one segment.

2.3. Noise

Noise stimuli are generated by stochastic generators. The most common is the Ornstein–Uhlenbeck process `ou`, a Gaussian noise with a given mean, standard deviation, and correlation time. It is often used to mimic the synaptic bombardment a neuron receives *in vivo*:

```
1s  dc(0)
10s ou(mean=100, sd=50, tau=5ms)
1s  dc(0)
```

Each time this file is played, the noise is a new realisation with the same statistics. To present exactly the same noise again, give it a seed:

```
10s  ou(mean=100, sd=50, tau=5ms, seed=17)
```

Any segment with `seed=17` and the same parameters produces the same samples, today or in ten years, on any computer (identical to the last digit of floating point in the strictest case, Sec. 11.5). Even without a seed, the seed that was actually used is stored with the data, so no realisation is ever lost (Sec. 11).

Other noise generators are `wnoise` (Gaussian white noise), `wnoise` (uniform white noise), and `cnoise` (noise with a $1/f^\alpha$ spectrum).

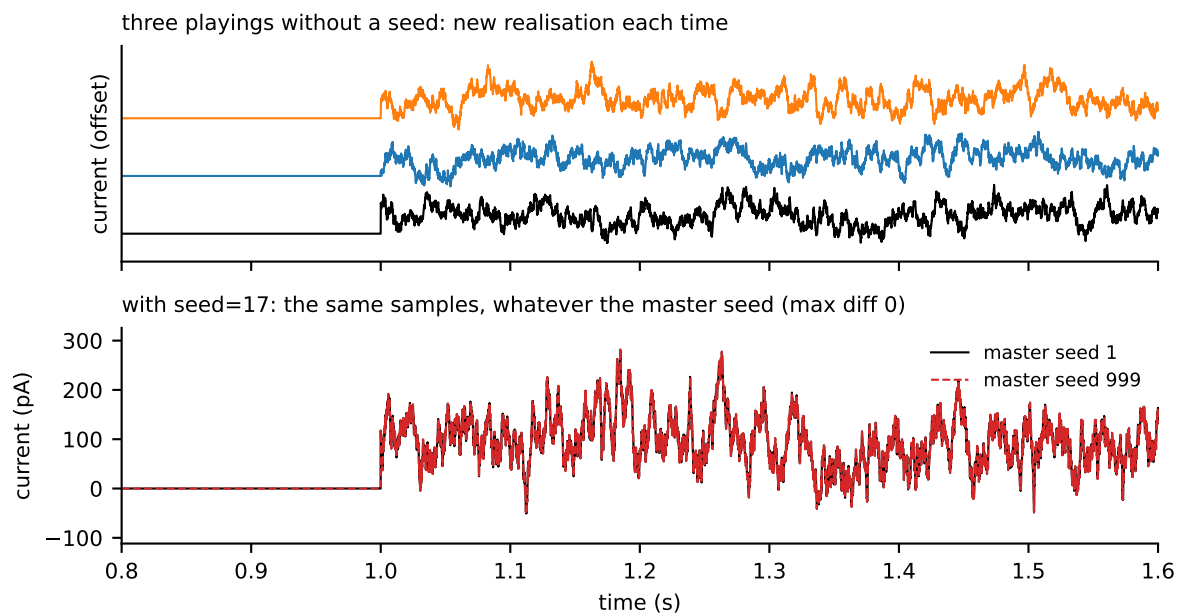


Figure 2.3.: Top: three playings of the OU waveform without a seed (traces offset for clarity) give three realisations with the same statistics. Bottom: with `seed=17`, two playings with different master seeds give identical samples (the dashed trace covers the solid one; the largest difference is printed in the title).

2.4. Pulse trains

Trains of brief pulses are used to evoke spikes or to stimulate synapses. The generator `pulses` covers regular and random trains:

```
1s    dc(0)
500ms pulses(amp=2000, rate=20Hz, width=1ms)          # 10 pulses
1s    dc(0)
2s    pulses(amp=2000, rate=20Hz, width=1ms, timing=poisson) # random times
```

The first train has pulses exactly every 50 ms, starting at the beginning of its segment. The second has the same mean rate, but the pulses occur at random times, as in a Poisson process. Every pulse of a train has exactly the same number of samples, which is what an experimenter expects (Sec. 9.5).

The shape of the pulses can be changed: `shape=exp` gives exponentially decaying pulses, `shape=biexp` synaptic-like pulses, `shape=biphasic` charge-balanced pulses for extracellular stimulation.

2.5. From description to samples

When a waveform is played or rendered, it is sampled at the rate of the set-up. Each segment covers the samples that fall within its time interval, and the boundaries are rounded so that rounding errors never accumulate (Sec. 8). In everyday use, none of this needs attention. It matters only when segments last a few samples or less, and then StimGen 2 warns.

A waveform can be checked and rendered from the command line. For example, with the illustrative tools of Table 14.2:

```
sg check step.sg
sg render --rate 20kHz step.sg -o step.sgb
sg render --rate 20kHz '500ms dc(0); 1s dc(300); 500ms dc(0)' -o step.sgb
```

The last line shows that a short waveform can also be written directly on the command line, with segments separated by `;`.

3. Combining and transforming waveforms

Elementary generators become much more useful when they are combined. This chapter shows how to add and multiply them, how to transform them, and how to build larger waveforms from smaller ones.

3.1. Arithmetic on generators

Within a segment, generators can be combined with `+`, `-`, `*`, and `/`, with the usual precedence and parentheses:

```
5s  sine(amp=50, freq=8Hz) + ou(mean=0, sd=20, tau=5ms)  # oscillation in noise
5s  100 + sine(amp=50, freq=8Hz)                          # sine with an offset
5s  (1 + 0.5*sine(1, 2Hz)) * sine(amp=80, freq=40Hz)     # amplitude modulation
5s  ou(mean=0, sd=1, tau=5ms) * (20 + 10*sine(1, 1Hz))   # noise of varying size
```

The operations act sample by sample. A number such as 100 or 0.5 is a constant generator, so a waveform is scaled by multiplying it by a number.

3.2. Transformations

Functions such as `abs`, `pos`, `clip`, and `pow` transform a waveform sample by sample (Table 10.3):

```
5s  abs(sine(4, 1Hz))                                     # full-wave rectified sine
5s  pos(sine(4, 1Hz))                                     # half-wave rectified sine
10s pos(ou(mean=10, sd=5, tau=3ms))                       # a conductance that is never negative
10s clip(ou(mean=0, sd=100, tau=5ms), -150, 150)
```

The positive part `pos` is handy in dynamic clamp, where a conductance must not become negative. Operations that are not defined, such as the square root of a negative value or a division by zero, stop the realisation with an error that names the segment and the sample. A stimulus is never silently altered.

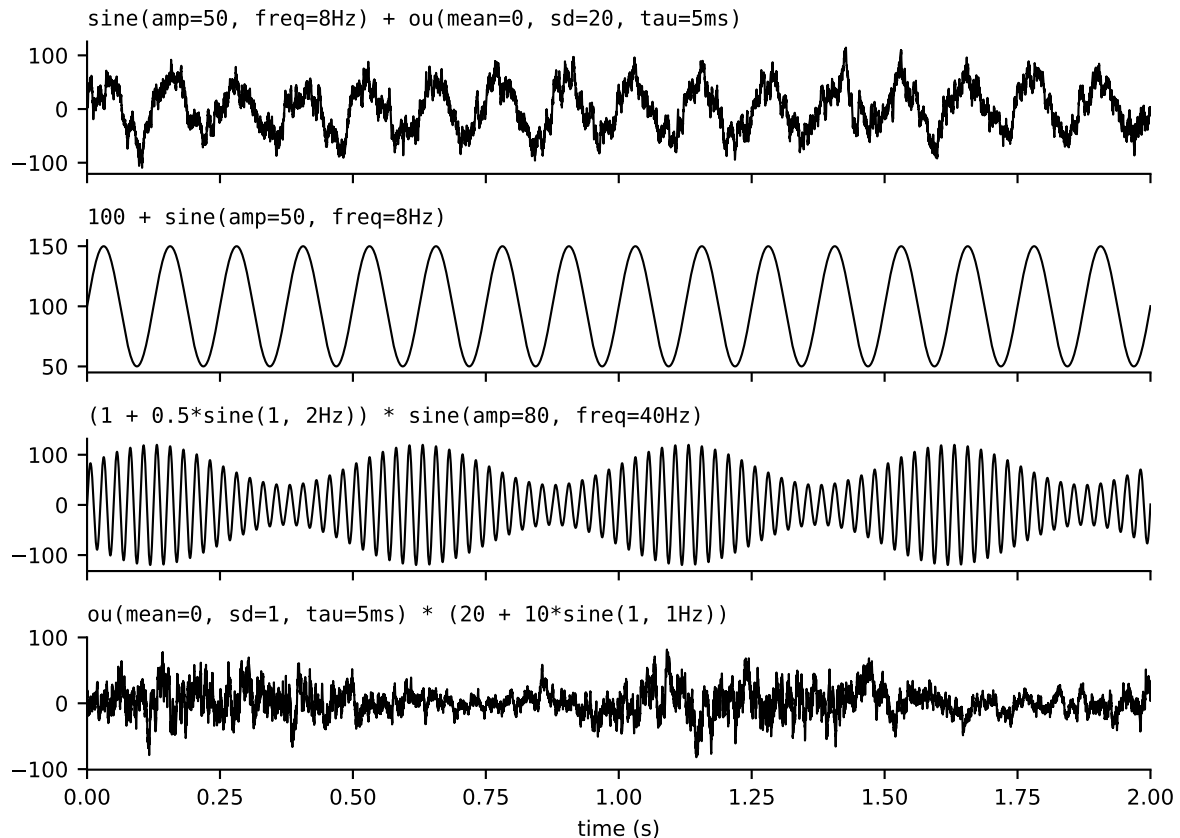


Figure 3.1.: The four segments above, first two seconds of each.

3.3. Ramps that continue

A ramp needs a start and an end. Often the start is simply where the previous segment ended, and the keyword `prev` says so:

```
1s    dc(-50)
2s    ramp(from=prev, to=200)      # from -50 to 200
1s    ramp(from=prev, to=0)        # back to 0
```

Since `from=prev` is the default of `ramp`, the second line can be shortened to `2s ramp(to=200)`. We recommend writing `from` explicitly whenever the ramp does not continue from the previous segment.

3.4. Blocks: waveforms inside waveforms

A sequence of segments in braces is a *block*. A block has a duration, the sum of its segments, and can be used inside an expression. This is how an envelope is applied to a carrier:

```
{ 1s ramp(0, 1) ; 3s dc(1) ; 1s ramp(1, 0) } * sine(amp=80, freq=8Hz)
```

The block lasts 5 s and describes a trapezoidal envelope. The sine has no duration of its own, so it takes the duration of the block and is multiplied by it, sample by sample. The result is a 5 s oscillation that fades in and out.

Two blocks combined in this way must have the same duration. Otherwise StimGen 2 reports an error before playing anything (Sec. 10.2).

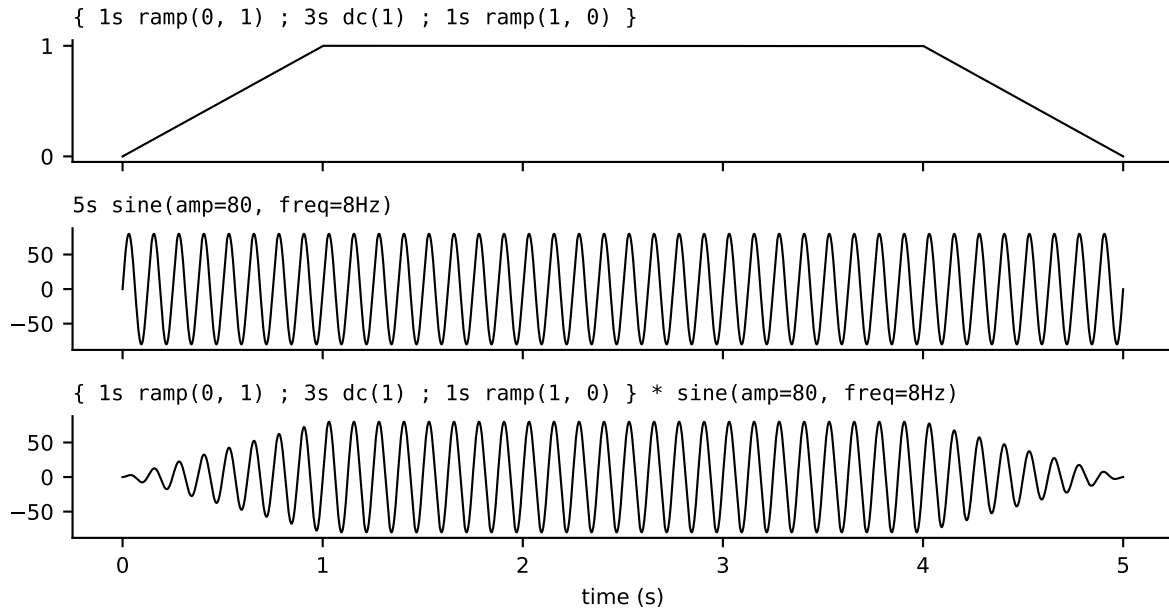


Figure 3.2.: A block as an envelope. Top: the block alone; middle: the carrier; bottom: their product, as written in the text.

3.5. Repeating a part of a waveform

A part of a waveform can be repeated with `repeat`:

```
1s dc(0)
repeat 10 { 2ms dc(1000) ; 48ms dc(0) }    # ten 2 ms pulses at 20 Hz
1s dc(0)
```

If the repeated part contains noise, each copy gets a different realisation, unless a seed is given. Note that `repeat` repeats a part of *one* waveform. Repeating a whole trial, with pauses in between, is done by a protocol (Sec. 5).

3.6. A larger example

The following waveform measures the response of a neuron to steps of increasing size, each embedded in background noise, inside a single trial:

```
# staircase in noise
1s dc(0)
{
```

```
    2s dc(100) ; 2s dc(200) ; 2s dc(300)
} + ou(mean=0, sd=30, tau=5ms, seed=3)
1s    dc(0)
```

The noise is added to the whole staircase at once and has a fixed seed, so every presentation of this file uses the same noise.

4. Multichannel stimuli and markers

Many experiments drive more than one output at a time: two cells recorded in a pair, a current command together with a light pulse, a stimulus together with a trigger for a camera. A *stimulus* file describes all outputs of one trial on a common time base.

4.1. Two outputs

This stimulus tests for a synaptic connection between two neurons. A train of brief pulses is injected into the first neuron, while the second is held at a constant current:

```
sg 2 stimulus

channel pre unit=pA {
    1s      dc(0)
    @train
    500ms    pulses(amp=2000, rate=20Hz, width=1ms)
    1s      dc(0)
}

channel post unit=pA {
    2.5s    dc(-50)
}
```

The first line, the *header*, states that the file holds a stimulus. Each **channel** has a name, a unit, and a waveform in braces. Both waveforms start at time zero and are sampled on the same grid. Here both last 2.5 s; if they did not, the shorter one would hold its *rest value* (0 unless stated) until the end, and StimGen 2 would warn.

Which physical output corresponds to **pre** and **post**, and with which amplifier gain, is not written in the file. It belongs to the configuration of the set-up, so the same file works on any rig (Sec. [12.2](#)).

4.2. Markers

The line **@train** *labels* the next segment. When the stimulus is played, StimGen 2 stores a marker named **pre.train** with the data, at the sample where the pulse train begins. Analysis scripts can then align responses to this marker instead of recomputing where the train started.

Markers can also be placed at explicit times:

```
marker flash at 1s, 1.5s
```

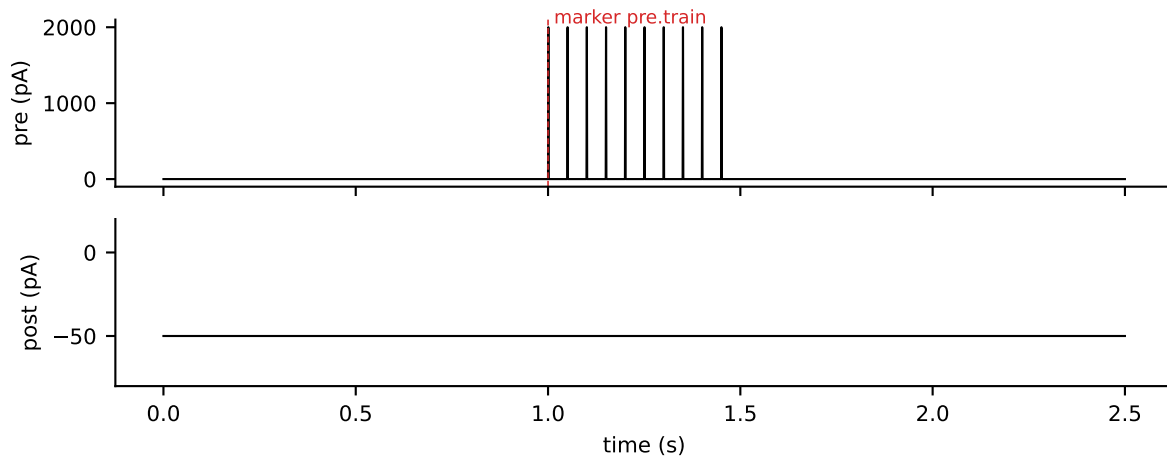


Figure 4.1.: The two channels of the pair stimulus, on a common time base. The dashed line is the marker `pre.train`, produced by the label `@train`.

4.3. Digital outputs

A digital output drives a trigger line, for instance to open a shutter or to start a camera. It is declared with `digital`, and its waveform must take only the values 0 and 1:

```
sg 2 stimulus
rate 20kHz

channel Iinj unit=pA {
  1s dc(0)
  2s ou(mean=150, sd=60, tau=5ms)
  1s dc(0)
}

digital camera {
  1s dc(0)
  2s pulses(amp=1, rate=100Hz, width=1ms)    # one frame trigger every 10 ms
  1s dc(0)
}
```

This file also fixes the sampling rate with `rate 20kHz`. Without that line, the rate of the set-up is used.

4.4. Same file, same noise?

Two channels may use the same waveform file:

```
channel A unit=pA use "noise.sg"
channel B unit=pA use "noise.sg"
```

Each channel then gets its *own* noise realisation, with the same statistics, because the random streams depend on the channel. To give two channels exactly the same samples, noise included, write

```
channel A unit=pA use "noise.sg"  
channel B unit=pA copy A
```

Both behaviours are useful, and the file states which one is meant.

5. Protocols: repetitions and sweeps

An experiment is a series of trials. A *protocol* file describes the series: which stimulus is played, which of its parameters change from trial to trial, how often each condition is repeated, in which order, and with which timing.

5.1. A family of current steps

The classic protocol for measuring the current–voltage relation and the firing rate of a neuron is a family of current steps:

```
sg 2 protocol
sweep    amp = from -300pA to 300pA step 50pA
repeat    3
order     shuffled-blocks
period    5s

stimulus {
    channel Iinj unit=pA {
        500ms dc(0)
        @step
        1s    dc($amp)
        500ms dc(0)
    }
}
```

The **sweep** line defines a *variable*, **amp**, which takes thirteen values from -300 to 300 pA. Inside the stimulus, **\$amp** stands for the current value. Each value is played three times (**repeat 3**), and the trials start every 5 s (**period 5s**).

With **order shuffled-blocks**, the thirteen steps are played in a random order, then again in another random order, and a third time in yet another order. Each step is thus played once before any is played twice, and slow drifts in the recording affect all amplitudes alike. The order is random but reproducible: it is derived from the protocol seed, which is recorded with the data. The other orders are **sequential**, **grouped**, and **shuffled** (Sec. 13.3). Fig. 5.1 shows the conditions and the order of the trials.

5.2. Several variables

With two **sweep** lines, every combination of values is played:

```
sweep amp  = [100pA, 200pA, 400pA]
sweep freq = [5Hz, 10Hz, 20Hz, 40Hz]
```

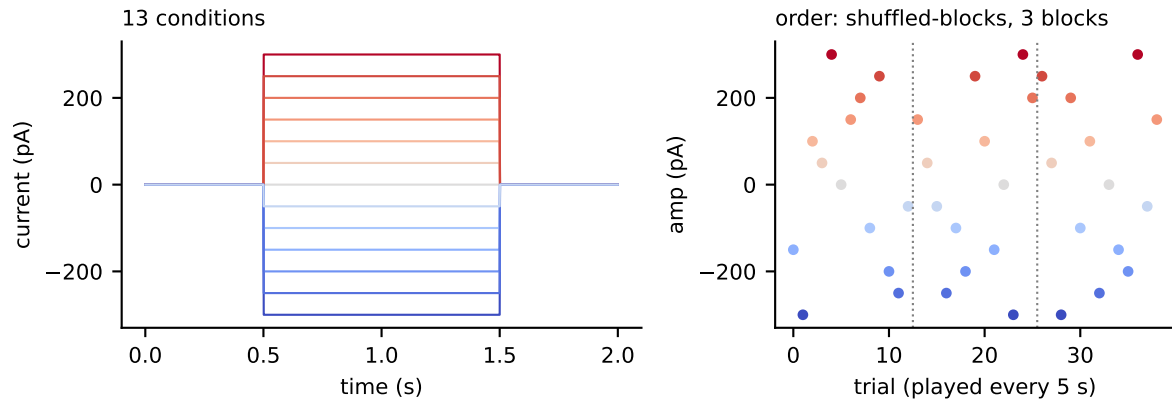


Figure 5.1.: The step protocol above, rendered by `sg render` (protocol seed 1). Left: the thirteen conditions. Right: the amplitude of each of the 39 trials in playing order; each block of thirteen trials (dotted lines) contains every amplitude once.

gives twelve conditions. A variable can also be computed from others with `let`, for example to keep the total charge of a pulse constant:

```
sweep width = [0.5ms, 1ms, 2ms]
let amp = 200pA * 1ms / width      # 400, 200, 100 pA
```

Variables can be used for any number in the stimulus, including durations, as in `$width` `dc($amp)`, and `$( ... )` holds an expression, as in `dc($(amp / 2))`. They exist only in protocols; waveform and stimulus files never contain them, which keeps those files simple to read.

5.3. Frozen noise and reliability

To measure how reliably a neuron reproduces its spike times, the same noise realisation is presented many times. With a protocol, this takes one line:

```
sg 2 protocol
repeat 50
gap 2s
noise per-condition

stimulus {
  channel Iinj unit=pA {
    500ms dc(0)
    10s ou(mean=150, sd=80, tau=5ms)
    500ms dc(0)
  }
}
```

`noise per-condition` gives all repetitions of a condition the same realisation. With the default, `noise per-trial`, every trial would receive fresh noise. With several noise levels as a sweep, each level would have its own frozen realisation. Here the trials are separated by a pause of 2 s after each one ends (`gap 2s`), rather than starting at fixed intervals.

5.4. Protocols and directories

A protocol can be *expanded* into a directory that contains one plain `.sg` file per trial and an index file that lists them in order, with their seeds and timing:

```
sg expand steps.sg steps/
```

Playing the directory is exactly the same as playing the protocol. The directory is useful to inspect every single trial before an experiment, to modify one by hand, or to play stimuli prepared by other means; any directory of `.sg` files can be played as a protocol (Sec. [13.4](#)).

Without protocol files, a family of steps would typically be produced by a shell loop that writes one file per amplitude:

```
for amp in $(seq -300 50 300); do
    echo "500ms dc(0); 1s dc($amp); 500ms dc(0)" > steps/step_${amp}.sg
done
```

This works, but the loop, the number of repetitions, the order, the interval, and the seed then live outside the data and are easily lost. The protocol file keeps them in one short text, stored with every recording.

Part II.

Reference specification

6. Conventions

This part of the document is normative. It defines what a StimGen 2 description means, independently of any particular implementation. Two conforming implementations **MUST** produce the same sequence of samples from the same description, the same sampling rate, and the same master seed, within the reproducibility levels defined in Sec. 11.

6.1. Requirement levels

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are used as defined in RFC 2119 (Bradner 1997). In short, **MUST** marks an absolute requirement for conformance, **SHOULD** marks a recommendation that may be ignored only for a documented reason, and **MAY** marks a truly optional feature.

Text inside boxes labelled *Note* or *Design rationale* is informative. It explains or motivates the normative text, but it adds no requirement.

6.2. Mathematical notation

We use the symbols in Table 6.1 throughout. Time is always a real number in seconds, unless a unit is written explicitly.

Table 6.1.: Notation used in the reference specification.

Symbol	Meaning
t	global time, measured from the start of the waveform (s)
u	local time, measured from the start of the current segment (s)
T, T_i	duration of a segment, of the i -th segment (s)
τ_i	start time of segment i , $\tau_i = \sum_{j < i} T_j$ (s)
$f_s, \Delta t$	sampling rate (Hz) and sampling interval $\Delta t = 1/f_s$ (s)
k	global sample index, $k = 0, 1, \dots, N - 1$
n_i	index of the first sample of segment i
x_k	value of the k -th sample of a realised waveform
g	a generator, i.e. a function of local time and duration (Sec. 7)
θ	the parameter vector of a generator
ξ	a pseudo-random variate (Gaussian with zero mean and unit variance unless stated)
$[x]_+$	positive part, $\max(x, 0)$
$\text{frac}(x)$	fractional part, $x - \lfloor x \rfloor$

Symbol	Meaning
$\mathbb{1}[\cdot]$	indicator function: 1 if the condition holds, 0 otherwise

Intervals are written with brackets and parentheses: $[a, b)$ contains a but not b . All segment intervals are half-open in this way (Sec. 8).

6.3. Units

6.3.1. Base units

Every parameter of a generator has a *dimension*, declared in its card in Sec. 9. The dimensions and their canonical units are:

Table 6.2.: Dimensions and unit suffixes.

Dimension	Canonical unit	Accepted suffixes
time	s	s, ms, us, ns, min
frequency	Hz	Hz, kHz, MHz
phase (angle)	rad	rad, deg
fraction	1	none, or % (divides by 100)
count	1	none (integer)
number	1	none (dimensionless real)
amplitude	channel unit	none, or a physical unit (Table 14.1)

Some parameters are not numbers: a *keyword* takes one of a listed set of names (e.g. `law=exp`), a *string* is quoted text (e.g. a file path), and a *list* is a bracketed, comma-separated sequence of values of one dimension (e.g. `times=[10ms, 25ms, 70ms]`).

A numeric literal without a suffix is interpreted in the canonical unit of the dimension of the field where it appears. Thus `tau=5` means 5 s, and a user who means 5 ms MUST write `tau=5ms`. The one exception is the duration of a segment, which MUST carry a time unit (Sec. 14).

A suffix whose dimension does not match the field MUST be rejected. Writing `freq=10ms` is an error, not a request for a period of 10 ms.

6.3.2. Amplitudes

The waveform layer is dimensionless with respect to amplitude. A waveform describes numbers, and the *stimulus* layer (Sec. 12) states what those numbers are: picoamperes injected in current clamp, millivolts in voltage clamp, nanosiemens of a dynamic-clamp conductance, or volts sent to an LED driver. This keeps the same waveform reusable across contexts.

An amplitude literal MAY carry a physical unit (e.g. `amp=300pA`). In that case the stimulus layer MUST check that the unit is compatible with the unit declared for the channel, and convert it. A waveform with amplitude units cannot be realised on a channel of a different dimension (for example, `pA` on a channel declared in `mV`).

6.4. Numbers

Numeric literals are decimal, with optional sign, fraction, and exponent (e.g. -2, 0.5, 1e-3). Durations and boundaries are computed in exact decimal arithmetic where Sec. 8 requires it. All other arithmetic is IEEE 754 double precision.

6.5. Files

A StimGen 2 description is stored as a plain text file, encoded in UTF-8, with the extension `.sg`. A `.sg` file holds one waveform, one stimulus, or one protocol (Sec. 7.1). Its first non-comment line, the *header*, states which, e.g. `sg 2 stimulus`. The header MAY be omitted in a file that holds a waveform (Sec. 14). The same text can also be given directly on the command line.

6.6. Conformance

A *description* is conforming if it parses under the grammar of Sec. 14 and passes the static checks of this part: known generators, parameters within their domains, matching dimensions, and matching durations in pointwise combinations (Sec. 10).

An *implementation* is conforming if, for every conforming description, it either produces the samples defined by this specification or reports an error in the cases this specification declares erroneous. An implementation MUST NOT silently replace an erroneous value (such as a non-finite sample) with another one.

7. The abstract model

This chapter defines the objects that a StimGen 2 description denotes. The definitions are purely mathematical. How these objects are written as text is the subject of Sec. 14, and how they are turned into samples is the subject of Sec. 8. We use a light preview of the syntax in the examples, so that the reader can connect the two.

7.1. Three layers

A description has three layers, each built on the previous one:

1. **Waveform.** One channel, one trial. A waveform is a real-valued function of time on a finite interval. It is written as a sequence of segments, and each segment is built from primitive generators by pointwise algebra. The waveform layer has no variables and no control flow.
2. **Stimulus.** Several waveforms played simultaneously on named output channels with a shared time base, together with markers (digital events) and the physical units of each channel (Sec. 12).
3. **Protocol.** An ordered collection of stimuli, with repetitions, inter-trial intervals, ordering rules, and parameter sweeps (Sec. 13). The protocol layer is the only place where variables exist.

Design rationale

Keeping the layers separate keeps each one simple. A waveform remains a short declarative text, readable at a glance, while the protocol layer carries all the logic that changes from one trial to the next. The protocol layer is defined by its expansion into a directory of plain `.sg` files (Sec. 13), so a protocol can always be inspected, or replaced, by an ordinary folder of stimuli.

7.2. Generators

A *generator* is a rule that produces a value for every instant of a segment, once the duration of the segment is known.

Definition 1 (deterministic generator). A deterministic generator is a function

$$g : (u, T; \theta) \mapsto g(u, T; \theta) \in \mathbb{R}, \quad 0 \leq u < T, \quad (7.1)$$

where u is local time, $T > 0$ is the duration of the segment that hosts the generator, and θ is a vector of parameters fixed when the generator is written.

Most generators ignore T . A sine wave, for example, is $g(u) = a \sin(2\pi f u + \varphi) + c$. Some generators need T : a linear ramp from a to b is $g(u, T) = a + (b - a) u / T$, and a chirp sweeps its

frequency from f_0 to f_1 over the whole segment. We call these *duration-aware*. The catalogue in Sec. 9 marks them.

Definition 2 (stochastic generator). A stochastic generator is a family of random functions, i.e. a stochastic process $\{G(u)\}_{0 \leq u < T}$ whose law depends on T , θ , and, for some generators, on the sampling interval Δt . A *realisation* of a stochastic generator is one sample path of that process. A stochastic generator becomes a definite function only once a random stream is attached to it (Sec. 11).

A *scalar* constant c is the deterministic generator $g(u) = c$. This lets us write `2 * sine(1, 5Hz)` without a separate scaling operation.

7.3. Timed waveforms

Definition 3 (timed waveform). A timed waveform is a pair $w = (T, \phi)$, where $T \geq 0$ is its duration and $\phi : [0, T) \rightarrow \mathbb{R}$ is its value as a function of time.

A generator becomes a timed waveform when it is given a duration. We call this operation *binding* and write it $g \upharpoonright T$:

$$g \upharpoonright T = (T, u \mapsto g(u, T; \theta)). \quad (7.2)$$

In the text syntax, binding is written by placing the duration before the generator:

```
5s sine(amp=3, freq=1Hz)
```

A *segment* is a timed waveform that appears as one element of a sequence.

7.4. Concatenation

Definition 4 (concatenation). The concatenation of two timed waveforms $w_1 = (T_1, \phi_1)$ and $w_2 = (T_2, \phi_2)$ is

$$w_1 ; w_2 = \left(T_1 + T_2, t \mapsto \begin{cases} \phi_1(t) & 0 \leq t < T_1, \\ \phi_2(t - T_1) & T_1 \leq t < T_1 + T_2. \end{cases} \right) \quad (7.3)$$

Concatenation is associative, $(w_1 ; w_2) ; w_3 = w_1 ; (w_2 ; w_3)$, and the empty waveform $\varepsilon = (0, \emptyset)$ is its identity. Timed waveforms under concatenation therefore form a monoid, and a sequence $w_1 ; w_2 ; \dots ; w_n$ needs no parentheses. Its duration is additive,

$$T_{\text{total}} = \sum_{i=1}^n T_i, \quad (7.4)$$

which also determines the length of the recording.

Each segment keeps its own local time. Inside segment i , local time is $u = t - \tau_i$, where $\tau_i = \sum_{j < i} T_j$ is the start of the segment. A periodic generator therefore restarts its phase at the beginning of each segment, unless it is told to follow global time instead (Sec. 8.5).

A sequence written in braces is itself a timed waveform and can be used anywhere a timed waveform is expected:

```
{ 100ms dc(0) ; 200ms ou(mean=2, sd=0.5, tau=1ms) ; 100ms dc(0) }
```

7.5. Pointwise combination

Waveforms of equal duration can be combined sample by sample. Let $\circ$ be one of the operations $+$, $-$, $\times$, $\div$.

Definition 5 (pointwise combination). For timed waveforms of equal duration,

$$(T, \phi_1) \circ (T, \phi_2) = (T, u \mapsto \phi_1(u) \circ \phi_2(u)). \quad (7.5)$$

For generators, $(g_1 \circ g_2)(u, T) = g_1(u, T) \circ g_2(u, T)$. When a timed waveform is combined with a generator, the generator is first bound to the duration of the timed waveform:

$$(T, \phi) \circ g = (T, \phi) \circ (g \upharpoonright T). \quad (7.6)$$

Combining two timed waveforms of different durations is an error.

These three rules give the language its expressive power without extra syntax. An envelope can be applied to a whole sequence, because the sequence is a timed waveform and the carrier is a generator:

```
{ 1s ramp(0, 1) ; 3s dc(1) ; 1s ramp(1, 0) } * sine(amp=2, freq=8Hz)
```

Here the braces have duration 5 s, so the sine generator is bound to 5 s and multiplied, point by point, by the trapezoidal envelope.

Unary maps such as absolute value or rectification act in the same pointwise way, $(T, \phi) \mapsto (T, h \circ \phi)$ for a real function h . They are listed in Sec. 10.

7.6. Context: the previous value

The value of a segment depends only on its own definition, with one exception. A generator parameter may be given the special value `prev`, which stands for the *last realised sample of the preceding segment in the same sequence* (0 if there is no preceding segment). The common use is a ramp that starts where the previous segment ended:

```
2.5s  dc(-1)
5s    ramp(from=prev, to=4)
```

`prev` is the only context-dependent construct of the waveform layer. It is defined on realised samples (rather than on the continuous function) so that it has a meaning also after a stochastic segment. Its exact definition, including its behaviour inside pointwise combinations, is given in Sec. 10.6.

7.7. Meaning of a description

We write $\llbracket D \rrbracket$ for the meaning of a description D .

- If D contains only deterministic generators, $\llbracket D \rrbracket$ is one timed waveform (T, ϕ) , a definite function of continuous time.
- If D contains stochastic generators, $\llbracket D \rrbracket$ is a random timed waveform: a probability distribution over functions. Given a master seed s , $\llbracket D \rrbracket_s$ is one realisation, and it is unique (Sec. 11).

The sampling operator of Sec. 8 turns $\llbracket D \rrbracket_s$ into a finite sequence of numbers $x_0, \dots, x_{N-1}$ for a given sampling rate.

Note

What independence of the sampling rate means. A StimGen 2 description does not depend on the waveform length, the sampling interval, or the D/A resolution. This holds exactly for the deterministic part: the description denotes a function of continuous time, and changing f_s only changes where that function is sampled. For stochastic generators it holds in distribution, not sample by sample. An Ornstein–Uhlenbeck realisation sampled at 10 kHz and one sampled at 20 kHz have the same mean, variance, and autocorrelation, but they are different sample paths. Discrete white noise is a stronger case. Its variance per sample is fixed, so its power spectral density, $\sigma^2 \Delta t$, scales with the sampling interval (Sec. 9.6.2).

7.8. Summary of the objects

Table 7.1.: The objects of the model.

Object	Has duration	Built from	Example
generator	no	primitives, scalars, operators	<code>sine(1, 5Hz) + 0.5</code>
segment	yes	duration + generator, or a block	<code>5s sine(1, 5Hz)</code>
sequence (block)	yes	segments joined by ;	<code>{ 1s dc(0) ; 2s dc(1) }</code>
waveform	yes	one top-level sequence	a <code>.sg</code> file
stimulus	yes	waveforms on channels + markers	Sec. 12
protocol	per trial	stimuli + repetitions + sweeps	Sec. 13

8. Sampling and quantisation

A description denotes a function of continuous time (Sec. 7). A D/A converter needs a finite sequence of numbers. This chapter defines the *sampling operator* that maps one to the other. Without it, two implementations can disagree on the number of samples in a segment, on where a step occurs, or on the phase of a sine wave.

8.1. The sampling grid

Sampling takes a sampling rate $f_s > 0$ (Hz), with sampling interval $\Delta t = 1/f_s$. The sampling rate is not part of the waveform. It is supplied by the stimulus layer or by the program that realises the waveform.

Sample k of a waveform corresponds to global time $t_k = k \Delta t$, for $k = 0, 1, \dots, N - 1$. The samples are *point evaluations*: no filtering, averaging, or band-limiting is applied (Sec. 8.7).

8.2. Segment boundaries

Consider a sequence of segments with nominal durations $T_1, \dots, T_m$ and nominal start times $\tau_i = \sum_{j < i} T_j$, with $\tau_1 = 0$ and $\tau_{m+1} = T_{\text{total}}$.

Rule 1 (boundary index). The first sample of segment i has index

$$n_i = R(\tau_i f_s), \quad i = 1, \dots, m + 1, \quad (8.1)$$

where R rounds to the nearest integer, with ties rounded to the nearest even integer.

Rule 2 (membership). Sample k belongs to segment i if and only if $n_i \leq k < n_{i+1}$. Segment i therefore contains $N_i = n_{i+1} - n_i$ samples, and the waveform contains $N = n_{m+1} = R(T_{\text{total}} f_s)$ samples.

Rule 3 (exact arithmetic). The products $\tau_i f_s$ MUST be evaluated so that the result of Eq. 8.1 equals the one obtained in exact arithmetic on the decimal literals of the description. An implementation MAY use rational or decimal arithmetic, or any other method with the same result. Binary floating point is not sufficient. For instance, $0.1 + 0.2$ is not 0.3 in IEEE 754, and at $f_s = 10$ Hz this shifts a boundary that falls exactly on a tie.

Three properties follow from these rules.

- **No drift.** Boundaries are rounded from cumulative time, not segment by segment. The error of every boundary is at most $\Delta t/2$, however long the sequence. If each segment were rounded on its own, the errors would add up.
- **No overlap and no gaps.** Every sample belongs to exactly one segment, because the intervals $[n_i, n_{i+1})$ partition $\{0, \dots, N - 1\}$.

- **Half-open segments.** The sample that falls exactly on a boundary belongs to the segment that *starts* there. A step from $\text{dc}(0)$ to $\text{dc}(1)$ at $t = 1$ s with $f_s = 1$ kHz has $x_{999} = 0$ and $x_{1000} = 1$.

Note

Example. Forty segments of 0.15 ms alternate between two levels, at $f_s = 10$ kHz ($\Delta t = 0.1$ ms). Each segment is 1.5 samples long. Rounding each segment separately would give 2 samples per segment, i.e. 80 samples (8 ms instead of 6 ms). Rules 1–2 give boundaries 0, 2, 3, 4, 6, 8, 9, 10, 12, The segments are 1 or 2 samples long (1.5 on average), and the total is exactly 60 samples. Fig. 8.1 shows the first eight segments as realised by **sg**.

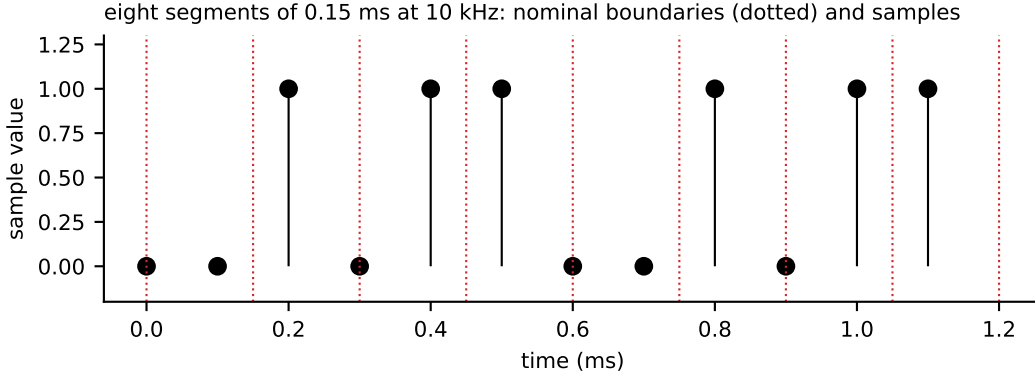


Figure 8.1.: Segment boundaries at 10 kHz for segments of 0.15 ms that alternate between 0 and 1. Dotted lines: nominal boundaries τ_i ; dots: samples. Each segment holds 1 or 2 samples, and no rounding error accumulates.

A segment may contain zero samples. This happens when its duration is shorter than about Δt , or when it is exactly zero (zero-duration segments are allowed, which is convenient in sweeps). An implementation **SHOULD** warn when a segment of positive duration contains no sample. A negative duration is an error.

8.3. Local time of samples

Inside segment i , sample k is evaluated at the *grid-aligned local time*

$$u_k = (k - n_i) \Delta t, \quad n_i \leq k < n_{i+1}, \quad (8.2)$$

and duration-aware generators receive the *realised duration*

$$\hat{T}_i = N_i \Delta t \quad (8.3)$$

in place of the nominal T_i . The value of sample k is then

$$x_k = g_i(u_k, \hat{T}_i; \theta_i). \quad (8.4)$$

In words, each segment is realised as if it started exactly on a sample and lasted an integer number of samples. Its first sample is always at local time $u = 0$, so a sine wave with zero phase starts at exactly zero. A ramp spans exactly the samples of its segment. The price is a timing offset between the continuous description and its realisation, at most $\Delta t/2$ at each boundary. This is the best that any sampled representation can do.

Design rationale

An alternative is to evaluate each sample at its exact continuous time, $x_k = g_i(t_k - \tau_i)$. Since $n_i \Delta t$ can be smaller than τ_i by up to $\Delta t/2$, the first sample of a segment would then be evaluated at a slightly negative local time, outside the domain of the generator, and a sine wave would not start at its written phase. Grid alignment avoids both problems and makes the realisation of every segment depend only on its own sample count.

8.4. End value of a segment

Each segment has an *end value* e_i . For a deterministic segment, it is the value the generator would take at the first sample after the segment,

$$e_i = g_i(\hat{T}_i, \hat{T}_i; \theta_i), \quad (8.5)$$

i.e. the right-hand continuation of the generator. A ramp from a to b has end value exactly b , even though its last sample is $a + (b - a)(N_i - 1)/N_i$. For a stochastic segment, the end value is its last realised sample. Combinations and unary maps propagate end values as defined in Sec. 10.6. The end value is what the keyword `prev` refers to.

Note

Because segments are half-open, a ramp never reaches its final value within its own samples. The next segment does. The sequence

```
1s  ramp(from=0, to=1)
1s  dc(1)
```

is continuous at the boundary in the sense that matters for a D/A converter: the sample sequence increases by equal steps up to and including the first sample of `dc(1)`.

8.5. Global clock

By default, local time restarts at zero at the beginning of every segment. Periodic generators (`sine`, `square`, `saw`, `triangle`) accept the optional parameter `clock`:

- `clock=local` (default): the generator is evaluated at u_k .
- `clock=global`: the generator is evaluated at global time $t_k = k \Delta t$, measured from the start of the waveform.

With `clock=global`, two segments of the same sine wave separated by a pause stay phase-locked to one continuous oscillation, as if the oscillation had kept running during the pause. This is useful, for example, when an oscillation is gated on and off by a sequence of segments.

8.6. Values outside the waveform

A waveform defines samples $x_0, \dots, x_{N-1}$ only. What the output does before the first and after the last sample is set by the *rest value* of the channel, defined in the stimulus layer (Sec. 12). The default rest value is 0.

8.7. Aliasing and edges

Point sampling does not remove frequencies above the Nyquist frequency $f_s/2$. Consequences for the user are:

- A sine or chirp whose frequency exceeds $f_s/2$ is realised as an alias at a lower frequency. An implementation **SHOULD** warn when any parameter of dimension *frequency* of a periodic generator exceeds $f_s/2$.
- The edges of square waves and pulses fall on the sampling grid. An edge at continuous time t^* is realised at the first sample with $t_k \geq t^*$, so a pulse of width w lasts $\lceil w/\Delta t \rceil$ or $\lfloor w/\Delta t \rfloor$ samples depending on where it starts. A pulse narrower than Δt can be lost entirely. Pulse generators therefore follow an explicit edge rule (Sec. 9.5).
- Discrete white noise has a flat spectrum up to $f_s/2$ by construction. Its continuous-time interpretation depends on f_s (Sec. 9.6.2).

Band-limited synthesis (e.g. of square waves) is outside the scope of this version.

8.8. Quantisation

The realised samples x_k are double-precision numbers in the units of the channel. Converting them into codes for a D/A converter is the last step of realisation, and it belongs to the stimulus layer, not to the waveform. For completeness, we specify it here.

Let the channel have an output gain γ (channel units per volt, e.g. 400 pA/V for a current-clamp command), and let the converter have B bits over the voltage range $[V_{\min}, V_{\max}]$. The code of sample k is

$$c_k = \text{clip}\left(\text{R}\left(\frac{x_k/\gamma - V_{\min}}{V_{\max} - V_{\min}} (2^B - 1)\right), 0, 2^B - 1\right). \quad (8.6)$$

The quantisation step in channel units is $q = \gamma(V_{\max} - V_{\min})/(2^B - 1)$. For example, a 16-bit converter over ± 10 V with $\gamma = 400$ pA/V gives $q \approx 0.12$ pA.

An implementation **MUST** report every sample that is clipped. It **MUST NOT** clip silently, because a clipped stimulus is a different stimulus from the one described.

9. Catalogue of primitive generators

This chapter lists the primitive generators of StimGen 2. Every primitive is described by a card with the same structure:

- a one-line description;
- a parameter table: name, dimension (Table 6.2), domain, and default value (a dash means the parameter is required);
- the definition, as a function of grid-aligned local time u and realised duration $\hat{T}$ (Sec. 8.3);
- the end value e (Sec. 8.4).

Parameters can be given by name, or by position in the order of the table. Parameters of dimension *amplitude* are in the units of the channel (Sec. 6.3.2). Every stochastic primitive also accepts the parameter `seed` (Sec. 11), which is not repeated in the tables.

Table 9.1.: Primitive generators.

Name	Kind	Duration-aware	Summary
<code>dc</code>	deterministic		constant level
<code>ramp</code>	deterministic	yes	linear ramp
<code>relax</code>	deterministic		exponential relaxation to a level
<code>sine</code>	deterministic, periodic		sinusoid
<code>square</code>	deterministic, periodic		square wave with duty cycle
<code>saw</code>	deterministic, periodic		asymmetric triangle (sawtooth)
<code>triangle</code>	deterministic, periodic		symmetric triangle
<code>chirp</code>	deterministic	yes	sinusoid with swept frequency
<code>biexp</code>	deterministic		difference of exponentials
<code>alpha</code>	deterministic		alpha function
<code>file</code>	deterministic		samples read from a file
<code>pulses</code>	deterministic or stochastic		train of pulses
<code>ou</code>	stochastic		Ornstein–Uhlenbeck process
<code>wnoise</code>	stochastic		Gaussian white noise
<code>unoise</code>	stochastic		uniform white noise
<code>cnoise</code>	stochastic	yes	power-law ($1/f^\alpha$) noise

9.1. Levels and ramps

9.1.1. `dc`: constant level

Parameter	Dimension	Domain	Default
<code>level</code>	amplitude	$\mathbb{R}$	—

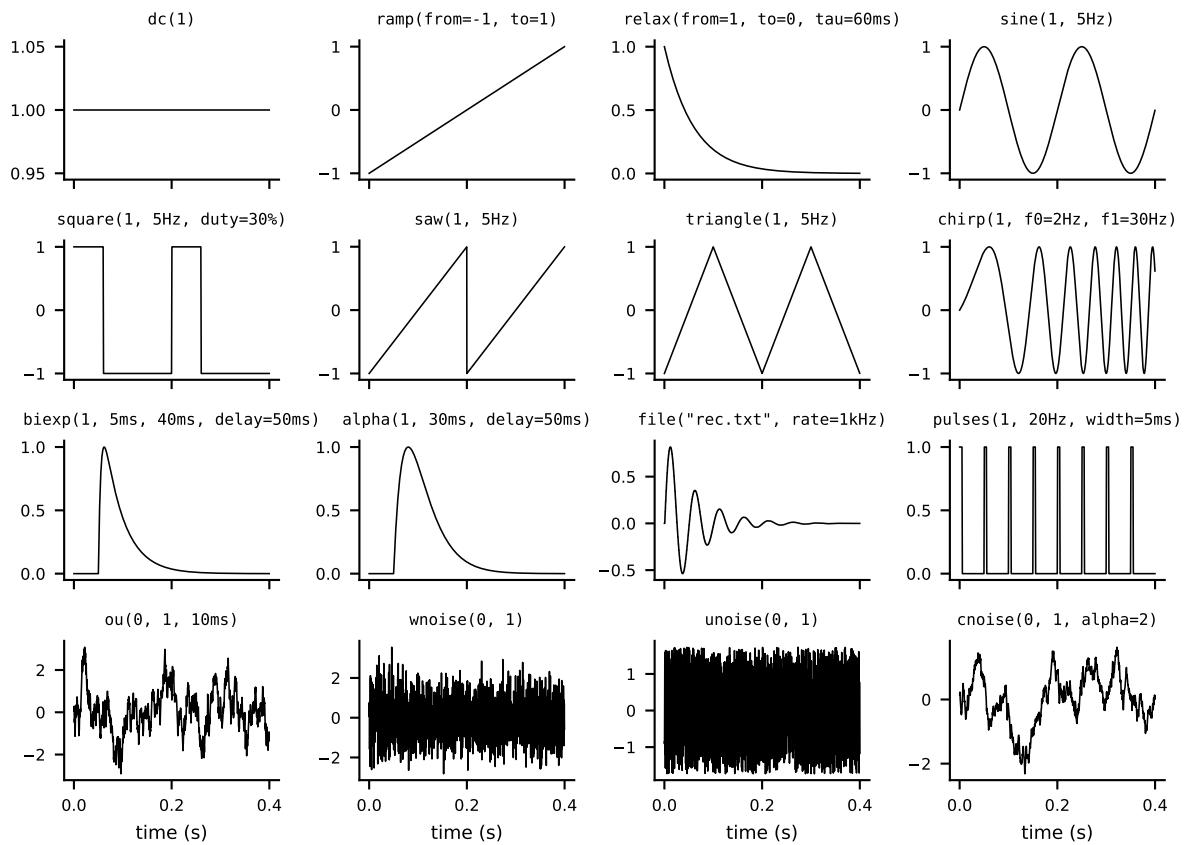


Figure 9.1.: Every primitive generator, rendered by **sg** as one segment of 400 ms at 5 kHz with the parameters shown (the **file** panel plays a damped oscillation stored as text at 1 kHz).

$$g(u) = \text{level}.$$

End value: `level`.

9.1.2. `ramp`: linear ramp

Parameter	Dimension	Domain	Default
<code>from</code>	amplitude	$\mathbb{R}$ or <code>prev</code>	<code>prev</code>
<code>to</code>	amplitude	$\mathbb{R}$	—

$$g(u, \hat{T}) = \text{from} + (\text{to} - \text{from}) \frac{u}{\hat{T}}. \quad (9.1)$$

End value: `to`. Duration-aware.

Because the default of `from` is `prev`, the short form `ramp(to=4)` starts from the end value of the preceding segment. (A single positional argument, as in `ramp(4)`, is `from`, and `to` is then missing.) We recommend writing `from` explicitly whenever the ramp is not meant to continue from the previous segment.

9.1.3. `relax`: exponential relaxation

Parameter	Dimension	Domain	Default
<code>from</code>	amplitude	$\mathbb{R}$ or <code>prev</code>	<code>prev</code>
<code>to</code>	amplitude	$\mathbb{R}$	—
<code>tau</code>	time	> 0	—

$$g(u) = \text{to} + (\text{from} - \text{to}) e^{-u/\tau}.$$

End value: $g(\hat{T})$. With `from=1`, `to=0` this is a decaying exponential; with `from=0`, `to=1` it is the charging curve of an RC circuit.

9.2. Periodic generators

The periodic generators share the parameters `phase` (rad, default 0), `offset` (amplitude, default 0), and `clock` (`local` or `global`, default `local`; see Sec. 8.5). For square, saw, and triangle waves we use the *cycle position*

$$p(u) = \text{frac}\left(\text{freq} \cdot u + \frac{\text{phase}}{2\pi}\right) \in [0, 1). \quad (9.2)$$

With `clock=global`, u is replaced by global time t_k in all formulas of this section. The end value of every periodic generator is its value at $u = \hat{T}$.

9.2.1. sine: sinusoid

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
freq	frequency	≥ 0	—
phase	phase	$\mathbb{R}$	0
offset	amplitude	$\mathbb{R}$	0

$$g(u) = \text{amp} \sin(2\pi \text{freq } u + \text{phase}) + \text{offset}.$$

amp is half the peak-to-peak amplitude.

9.2.2. square: square wave

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
freq	frequency	> 0	—
duty	fraction	$[0, 1]$	0.5
phase	phase	$\mathbb{R}$	0
offset	amplitude	$\mathbb{R}$	0

$$g(u) = \begin{cases} \text{offset} + \text{amp} & p(u) < \text{duty}, \\ \text{offset} - \text{amp} & \text{otherwise.} \end{cases} \quad (9.3)$$

The wave alternates between $\text{offset} \pm \text{amp}$, and the high level lasts the fraction **duty** of each period. Its mean is $\text{offset} + \text{amp} (2 \text{duty} - 1)$, which is zero only when $\text{duty} = 0.5$ and $\text{offset} = 0$.

9.2.3. saw: sawtooth

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
freq	frequency	> 0	—
duty	fraction	$[0, 1]$	1
phase	phase	$\mathbb{R}$	0
offset	amplitude	$\mathbb{R}$	0

$$g(u) = \text{offset} + \begin{cases} -\text{amp} + 2 \text{amp} \frac{p(u)}{\text{duty}} & p(u) < \text{duty}, \\ \text{amp} - 2 \text{amp} \frac{p(u) - \text{duty}}{1 - \text{duty}} & \text{otherwise.} \end{cases} \quad (9.4)$$

The wave rises linearly from $-\text{amp}$ to $+\text{amp}$ during the fraction **duty** of each period and falls back during the rest. It is the time integral, rescaled, of a zero-mean square wave with the same duty cycle, and its own mean is zero for any **duty**. With **duty** = 1 it is a rising sawtooth, with **duty** = 0 a falling sawtooth, and with **duty** = 0.5 a symmetric triangle. At $u = 0$ with zero phase, it starts at its minimum.

9.2.4. triangle: symmetric triangle

`triangle(amp, freq, phase, offset)` is a shorthand for `saw(amp, freq, duty=0.5, phase, offset)`.

9.2.5. chirp: swept-frequency sinusoid

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
f0	frequency	≥ 0	—
f1	frequency	≥ 0	—
law	keyword	linear , exp	linear
phase	phase	$\mathbb{R}$	0
offset	amplitude	$\mathbb{R}$	0

$$g(u, \hat{T}) = \text{amp} \sin(\Phi(u) + \text{phase}) + \text{offset},$$

where the instantaneous frequency $\Phi'(u)/2\pi$ moves from **f0** at $u = 0$ to **f1** at $u = \hat{T}$. For the linear law,

$$\Phi(u) = 2\pi \left(f_0 u + \frac{f_1 - f_0}{2\hat{T}} u^2 \right), \quad (9.5)$$

and for the exponential law, which requires $f_0, f_1 > 0$ and spends equal time in every octave,

$$\Phi(u) = 2\pi f_0 \hat{T} \frac{(f_1/f_0)^{u/\hat{T}} - 1}{\ln(f_1/f_0)} \quad (f_0 \neq f_1), \quad (9.6)$$

which reduces to the linear law when $f_0 = f_1$. The derivations are in Sec. B. End value: $g(\hat{T}, \hat{T})$. Duration-aware.

Chirps are the stimuli used to measure the impedance profile of neurons (ZAP stimuli) (Puil et al. 1986).

9.3. Synaptic-like waveforms

These waveforms take the value **offset** before **delay** and are defined in terms of the time since the delay, $v = u - \text{delay}$.

9.3.1. biexp: difference of exponentials

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
tau_rise	time	> 0	—
tau_decay	time	$> \mathbf{tau_rise}$	—
delay	time	≥ 0	0
offset	amplitude	$\mathbb{R}$	0

$$g(u) = \mathbf{offset} + \frac{\mathbf{amp}}{K} \left(e^{-v/\tau_d} - e^{-v/\tau_r} \right) \mathbb{1}[v \geq 0], \quad (9.7)$$

with $\tau_r = \mathbf{tau_rise}$, $\tau_d = \mathbf{tau_decay}$, and the normalisation

$$K = \left(\frac{\tau_r}{\tau_d} \right)^{\frac{\tau_r}{\tau_d - \tau_r}} - \left(\frac{\tau_r}{\tau_d} \right)^{\frac{\tau_d}{\tau_d - \tau_r}}, \quad (9.8)$$

which makes the peak value equal to **amp**. The peak occurs at $v^* = \frac{\tau_r \tau_d}{\tau_d - \tau_r} \ln(\tau_d / \tau_r)$. End value: $g(\hat{T})$.

9.3.2. alpha: alpha function

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
tau	time	> 0	—
delay	time	≥ 0	0
offset	amplitude	$\mathbb{R}$	0

$$g(u) = \mathbf{offset} + \mathbf{amp} \frac{v}{\tau} e^{1-v/\tau} \mathbb{1}[v \geq 0]. \quad (9.9)$$

The peak value **amp** is reached at $v = \tau$. This is the limit of **biexp** when **tau_rise** and **tau_decay** both tend to τ . End value: $g(\hat{T})$.

9.4. Arbitrary samples

9.4.1. file: samples from a file

Parameter	Dimension	Domain	Default
path	string	readable file	—
rate	frequency	> 0	—
column	count	≥ 1	1
interp	keyword	hold , linear	hold
extend	keyword	error , hold , zero , cycle	error

The file holds M numbers $y_0, \dots, y_{M-1}$, sampled at **rate**. It is a text file with one row per sample; **column** selects a column when there are several. Relative paths are resolved from the directory of the description. With $r = \text{rate}$ and $j = \lfloor ur \rfloor$,

$$g(u) = \begin{cases} y_j & \text{interp=hold,} \\ y_j + (ur - j)(y_{j+1} - y_j) & \text{interp=linear.} \end{cases}$$

When j falls beyond the last sample, **extend** decides: an error (default), the last value, zero, or a periodic repetition of the file. End value: g evaluated at $u = \hat{T}$ by the same rules, except that with **extend=error** the last sample is held, so that a file exactly as long as its segment is accepted.

This primitive is the escape hatch for waveforms that have no compact description, such as a recorded synaptic current played back in dynamic clamp. It gives up the main advantage of the language, which is that the description is short. The provenance record MUST include a cryptographic hash of the file (Sec. 15).

9.5. Pulse trains

9.5.1. pulses: train of pulses

A pulse train is the superposition of copies of a *kernel* h , one at each onset time $s_1 < s_2 < \dots$ inside the segment:

$$g(u) = \text{offset} + \text{amp} \sum_n h(u - s_n). \quad (9.10)$$

The onset times are regular, random (Poisson), or listed explicitly.

Parameter	Dimension	Domain	Default
amp	amplitude	$\mathbb{R}$	—
rate	frequency	> 0	— (unless times is given)
shape	keyword	square , biphasic , exp , alpha , biexp	square
width	time	> 0	— (for square , biphasic)
tau	time	> 0	— (for exp , alpha)
tau_rise , tau_decay	time	$0 < \tau_r < \tau_d$	— (for biexp)
timing	keyword	regular , poisson	regular
times	list of times	increasing, ≥ 0	none
delay	time	≥ 0	0
dead	time	$[0, 1/\text{rate})$	0
align	keyword	grid , exact	grid
offset	amplitude	$\mathbb{R}$	0

Kernels. With $w = \text{width}$ and the time v since onset:

Table 9.2.: Pulse kernels; every kernel has peak value 1.

shape	$h(v)$ for $v \geq 0$ (and $h = 0$ for $v < 0$)
square	$\mathbb{1}[v < w]$
biphasic	$\mathbb{1}[v < w] - \mathbb{1}[w \leq v < 2w]$ (each phase lasts w)
exp	$e^{-v/\tau}$
alpha	$(v/\tau) e^{1-v/\tau}$
biexp	$(e^{-v/\tau_d} - e^{-v/\tau_r})/K$, with K from Eq. 9.8

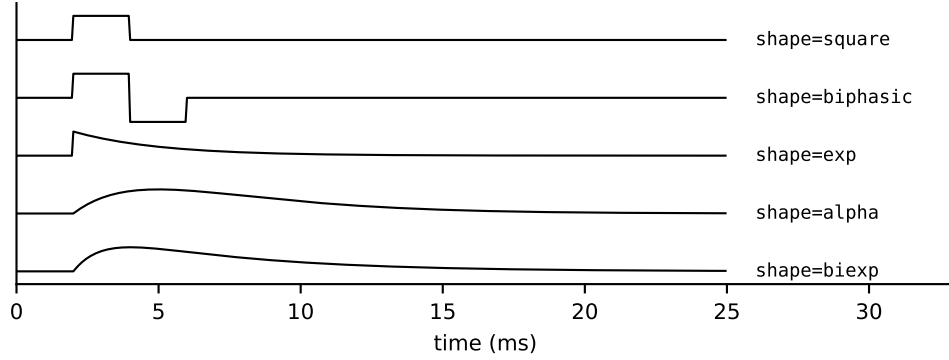


Figure 9.2.: The five kernels, each as a single pulse at 2 ms, rendered with `pulses(1, times=[2ms], shape=...)`.

Onset times.

- **timing=regular**: $s_n = \text{delay} + n/\text{rate}$, for $n = 0, 1, 2, \dots$. The first pulse starts at **delay**.
- **timing=poisson**: $s_1 = \text{delay} + I_1$ and $s_{n+1} = s_n + I_{n+1}$, where the intervals I_n are independent, each equal to **dead** plus an exponential variate with mean $1/\text{rate} - \text{dead}$. The mean rate is therefore exactly **rate**. With **dead** = 0, the onsets form a homogeneous Poisson process; a positive **dead** time is a refractory period that prevents pulses from overlapping. The pulse train is then a stochastic generator.
- **times=[...]**: the onsets are the listed local times, plus **delay**.

Only onsets with $s_n < \hat{T}$ are used. A pulse that starts inside the segment and would extend beyond its end is truncated; it does not spill into the next segment.

Overlap. Pulses are summed, as Eq. 9.10 states. Two overlapping square pulses give a value of $2 \cdot \text{amp}$. Use **dead** $\geq$ **width** to exclude overlaps.

Edges. With **align=grid** (default), each onset is moved to the nearest sample, $s_n \rightarrow R(s_n/\Delta t) \Delta t$, and the width of square and biphasic pulses is rounded to a whole number of samples, $\max(1, R(w/\Delta t))$. Every pulse then has exactly the same number of samples, which is what an experimenter expects from a train of identical stimuli. With **align=exact**, Eq. 9.10 is evaluated at the sample times without any rounding, so pulse widths can differ by one sample. An implementation SHOULD warn when $w < \Delta t$.

End value: **offset** plus the summed contribution of the pulses at $u = \hat{T}$. This is generally **offset** for square pulses and nonzero for exponential tails.

9.6. Noise processes

The noise generators below are stochastic. Their random variates come from the stream attached to the generator (Sec. 11). In the formulas, ξ_k denotes independent standard Gaussian variates and U_k independent uniform variates on $[0, 1)$, both indexed by the sample within the segment.

9.6.1. ou: Ornstein–Uhlenbeck process

Parameter	Dimension	Domain	Default
mean	amplitude	$\mathbb{R}$	—
sd	amplitude	≥ 0	—
tau	time	> 0	—
init	keyword or amplitude	stationary, mean, prev, or a value	stationary

The Ornstein–Uhlenbeck process (Uhlenbeck and Ornstein 1930) is the stationary Gaussian process with mean $\mu = \text{mean}$, standard deviation $\sigma = \text{sd}$, and autocorrelation

$$\text{Cov}(X(u), X(u+s)) = \sigma^2 e^{-|s|/\tau}.$$

Equivalently, it is the solution of the stochastic differential equation

$$dX = -\frac{X - \mu}{\tau} du + \sigma \sqrt{\frac{2}{\tau}} dW, \quad (9.11)$$

i.e. white noise passed through a first-order low-pass filter with time constant τ . Its one-sided power spectral density is $S(f) = 4\sigma^2\tau/(1 + (2\pi f\tau)^2)$.

Realisation. The samples MUST be generated with the exact update (Gillespie 1996)

$$x_{k+1} = \mu + (x_k - \mu)\rho + \sigma\sqrt{1 - \rho^2}\xi_{k+1}, \quad \rho = e^{-\Delta t/\tau}, \quad (9.12)$$

which reproduces the statistics of the continuous process at the sample times for any Δt , however large compared with τ . The first sample of the segment is $x_0 = \mu + \sigma\xi_0$ for **init=stationary**, μ for **init=mean**, the end value of the previous segment for **init=prev**, or the given value.

With **init=stationary**, the realisation is stationary from its first sample. With any other choice, the mean and variance relax towards their stationary values with time constants τ and $\tau/2$, respectively. End value: the last sample.

9.6.2. wnoise: Gaussian white noise

Parameter	Dimension	Domain	Default
mean	amplitude	$\mathbb{R}$	—
sd	amplitude	≥ 0	—

$$x_k = \mu + \sigma \xi_k.$$

The samples are independent. The variance per sample, σ^2 , is fixed, so the one-sided power spectral density, $2\sigma^2\Delta t$ on $[0, f_s/2]$, depends on the sampling rate. Doubling f_s at fixed **sd** halves the noise power per hertz. A user who wants a fixed spectral density S_0 should choose $\sigma = \sqrt{S_0 f_s/2}$. End value: the last sample.

wnoise is the limit of **ou** for $\tau \ll \Delta t$, where $\rho \rightarrow 0$ in Eq. 9.12.

9.6.3. **wnoise**: uniform white noise

Parameter	Dimension	Domain	Default
mean	amplitude	$\mathbb{R}$	—
sd	amplitude	≥ 0	—

$$x_k = \mu + \sigma\sqrt{12} \left(U_k - \frac{1}{2} \right).$$

The samples are independent and uniformly distributed on $[\mu - \sigma\sqrt{3}, \mu + \sigma\sqrt{3}]$, with mean μ and standard deviation σ . The spectral remarks for **wnoise** apply. End value: the last sample.

9.6.4. **cnoise**: power-law noise

Parameter	Dimension	Domain	Default
mean	amplitude	$\mathbb{R}$	—
sd	amplitude	≥ 0	—
alpha	number	≥ 0	1
fmin	frequency	≥ 0	$1/\hat{T}$
fmax	frequency	$\leq f_s/2$	$f_s/2$

A Gaussian process with power spectral density proportional to $f^{-\alpha}$ between **fmin** and **fmax**, and zero elsewhere. The value $\alpha = 0$ gives band-limited white noise, $\alpha = 1$ pink noise, and $\alpha = 2$ Brownian (“red”) noise.

Realisation. The whole segment is synthesised at once in the frequency domain (Timmer and König 1995). For the N_i samples of the segment and the frequencies $f_j = j/(N_i\Delta t)$, the Fourier coefficient at f_j has independent Gaussian real and imaginary parts with variance proportional to $f_j^{-\alpha}$ inside the band, and zero outside. An inverse discrete Fourier transform gives the time series, which is scaled so that its *expected* variance is σ^2 and shifted by μ . The exact procedure, including the treatment of the zero and Nyquist frequencies, is in Sec. B.

This generator is duration-aware and non-causal: every sample depends on the length of the segment and on random variates drawn for the whole segment. The realised standard deviation fluctuates around σ , more so for large α . End value: the last sample.

10. Algebra of waveforms

This chapter defines how generators and timed waveforms are combined into expressions: the operators, their precedence, the unary maps, the **repeat** construct, and the exact meaning of **prev**. The abstract objects were introduced in Sec. 7.

10.1. Expressions

An *expression* is built from the following operands:

- a primitive generator, e.g. `sine(amp=1, freq=5Hz)` (Sec. 9);
- a numeric constant, e.g. `0.5` or `-2`, which is the constant generator;
- a *block*, i.e. a sequence of segments in braces, e.g. `{ 1s dc(0) ; 2s dc(1) }`, which is a timed waveform;
- a **repeat** construct (Sec. 10.5), which is a timed waveform.

Operands are combined with the binary operators `+`, `-`, `*`, `/`, the unary minus, and the functions of Table 10.3. Round parentheses group subexpressions.

A *segment* is an expression with a duration. It is written either as a duration followed by an expression (`5s sine(1, 5Hz) + 0.5`), or as an expression that already has a duration (a block, or a combination containing a block). If both are given, they **MUST** be equal.

10.1.1. Precedence

Table 10.1.: Operator precedence.

Level	Operators	Associativity
1 (highest)	function application: primitives and maps, e.g. <code>abs(...)</code>	—
2	unary minus	—
3	<code>*</code> , <code>/</code>	left
4 (lowest)	<code>+</code> , <code>-</code>	left

These are the conventions of ordinary arithmetic. The expression `a + b * c` means $a + (b \times c)$, and `a - b - c` means $(a - b) - c$.

10.2. Duration rules

Every expression has a *kind*: either *generator* (no duration) or *timed* with a nominal duration T . The kind is determined statically, before any sample is computed, by the rules in Table 10.2.

Table 10.2.: Kinds of expressions; $\circ$ is any binary operator.

Expression	Kind
primitive, constant	generator
block { s_1 ; ... ; s_n }	timed, $T = \sum T_{s_i}$
repeat n { ... }	timed, n times the duration of the block
$e_1 \circ e_2$, both generators	generator
$e_1 \circ e_2$, one timed with T , one generator	timed, T
$e_1 \circ e_2$, both timed with $T_1 = T_2 = T$	timed, T
$e_1 \circ e_2$, both timed with $T_1 \neq T_2$	error
$h(e)$, unary map	same kind as e

Durations are compared exactly, on their decimal values (Sec. 8).

When a generator is combined with a timed operand, it is bound to the same interval as that operand: its local time starts with the first sample of the operand, and its realised duration is that of the operand.

10.2.1. Nested sequences and sampling

Blocks can be nested. Segment boundaries are always computed from absolute nominal start times with Rule 1 of Sec. 8.2, whatever the nesting depth. A segment nested inside a block that starts at τ_B and begins at offset δ within the block has absolute start time $\tau_B + \delta$. Two operands of a pointwise combination start at the same absolute time and have the same nominal duration, so they always contain the same samples.

10.3. Binary operators

Binary operators act sample by sample, Eq. 7.5. Arithmetic follows IEEE 754 double precision.

Rule 4 (finite samples). Every realised sample **MUST** be a finite number. If an operation produces an infinity or a NaN (for example, a division by a waveform that passes through zero), realisation **MUST** stop with an error that identifies the segment and the sample. Since stochastic waveforms can take any value, this check cannot be done statically in general.

Two binary functions complete the operators: $\min(\mathbf{e1}, \mathbf{e2})$ and $\max(\mathbf{e1}, \mathbf{e2})$, which take the smaller or larger of the two values at each sample. They follow the same duration rules.

10.4. Unary maps

A unary map applies a real function h to every sample of its argument. The maps of Table 10.3 are defined.

Table 10.3.: Unary maps.

Map	Definition	Domain of the argument
<code>abs(e)</code>	$ x $	$\mathbb{R}$
<code>pos(e)</code>	$[x]_+ = \max(x, 0)$	$\mathbb{R}$
<code>pow(e, k)</code>	x^k	$x \geq 0$, or any x if k is an integer; $x \neq 0$ if $k < 0$
<code>spow(e, k)</code>	$\text{sign}(x) x ^k$	$\mathbb{R}$; $x \neq 0$ if $k < 0$
<code>sqrt(e)</code>	$\sqrt{x}$	$x \geq 0$
<code>exp(e)</code>	e^x	$\mathbb{R}$
<code>log(e)</code>	$\ln x$	$x > 0$
<code>clip(e, lo, hi)</code>	$\min(\max(x, lo), hi)$	$\mathbb{R}$; requires $lo \leq hi$

A sample outside the domain of a map is an error, by Rule 4. The map **spow** (signed power) is defined everywhere and is the safe choice for compressing or expanding a waveform that takes both signs.

10.5. Repetition inside a waveform

The construct

```
repeat n { s1 ; s2 ; ... }
```

with a non-negative integer n , is a timed waveform equal to n copies of the block, concatenated. It is pure shorthand. For example, ten pulses of 1 ms at 10 Hz can be written

```
repeat 10 { 1ms dc(1) ; 99ms dc(0) }
```

Every copy is a separate part of the waveform. Stochastic generators in different copies therefore receive different random streams and produce different realisations, unless their seed is fixed explicitly (Sec. 11). The keyword **prev** in the first segment of a copy refers to the end value of the previous copy.

Note

repeat repeats a *part of one waveform*. Repeating a whole stimulus several times, with pauses between trials, is a matter for the protocol layer (Sec. 13), and it produces separate recordings.

10.6. The previous value

10.6.1. End values of expressions

The end value of a segment was defined for primitives in Sec. 8.4. It extends to every expression recursively:

- $E(c) = c$ for a constant c ;

- $E(g)$ for a primitive g is given in its card (Sec. 9);
- $E(e_1 \circ e_2) = E(e_1) \circ E(e_2)$ for a binary operator or binary function;
- $E(h(e)) = h(E(e))$ for a unary map;
- $E(\{s_1; \dots; s_n\}) = E(s_n)$, and E of a **repeat** is the end value of its last copy (0 copies: see below);
- for a stochastic primitive, E is its last realised sample, or, if the segment has no sample, the value of **prev** where it appears.

If evaluating an end value violates Rule 4 (for example, a division by zero at $u = \hat{T}$), the end value is undefined, and it is an error to refer to it with **prev**.

10.6.2. Resolution of **prev**

The keyword **prev** may appear as the value of any parameter of dimension *amplitude*. It is resolved as follows.

1. Inside a segment s_i of a sequence, **prev** is the end value $E(s_{i-1})$ of the preceding segment *of the same sequence*.
2. Inside the first segment of a block, **prev** is the value of **prev** at the place where the block appears (rule 1 or 2 applied to the enclosing sequence). Likewise for a **repeat** with zero copies, whose end value is that same **prev**.
3. In the first segment of the waveform, **prev** is 0.

All occurrences of **prev** within one segment, including those inside every operand of a combination, refer to the same value. The value depends only on segments that come earlier in time, so it is always well defined.

10.7. Static checks

Before computing any sample, an implementation **MUST** check that:

- every primitive and map is known, and every parameter is known, given at most once, and within its domain;
- every literal has a unit compatible with the dimension of its field (Sec. 6.3);
- every combination of timed operands has matching durations (Table 10.2), and every segment has a duration;
- every duration is non-negative and every **repeat** count is a non-negative integer.

Errors found by these checks **MUST** be reported with the position of the offending text. Only Rule 4 errors and file errors can occur during realisation.

10.8. Algebraic laws (informative)

The following identities hold for timed waveforms and are useful to implementers, for example to simplify or normalise expressions.

- For timed waveforms of equal duration, $+$ and $*$ are associative and commutative, $*$ distributes over $+$, and the constants 0 and 1 are their identities. Pointwise operations thus form a commutative ring, up to the usual rounding of floating point.
- Concatenation is associative, with the empty waveform as identity (Sec. 7.4).

- **Interchange law.** If $T_a = T_c$ and $T_b = T_d$, then $(a; b) \circ (c; d) = (a \circ c); (b \circ d)$. A combination of two sequences with the same segmentation can therefore be rewritten as a sequence of combinations, and vice versa.

Rewriting an expression gives the same deterministic samples, but it can change the addresses of stochastic generators and therefore their realisations, though not their statistics (Sec. 11).

10.9. Examples

The following segments illustrate the operators. Each line is one segment.

```
5s  sine(1, 1Hz) + ou(mean=0, sd=0.2, tau=5ms)    # sine plus noise
5s  sine(1, 1Hz) * ou(mean=0, sd=2, tau=5ms)      # noise with a modulated amplitude
5s  abs(sine(4, 1Hz))                             # full-wave rectification
5s  pos(sine(4, 1Hz))                             # half-wave rectification
30s ramp(from=50, to=300)                         # ramp from 50 to 300
5s  (1 + 0.5*sine(1, 2Hz)) * sine(1, 40Hz)        # amplitude modulation
10s pos(ou(mean=10, sd=5, tau=3ms))               # non-negative conductance
```

11. Stochastic semantics and reproducibility

Noise stimuli are central to cellular electrophysiology. They are used to probe input–output transfer, spike-time reliability, and synaptic integration, and many of these experiments rely on presenting the *same* noise realisation more than once (“frozen noise”). This chapter defines exactly which realisation a description produces, so that any realisation can be regenerated later from the description and a seed.

11.1. Process and realisation

A description that contains stochastic generators denotes a *random* waveform, i.e. a probability law over waveforms (Sec. 7.7). The statistics of this law (mean, variance, autocorrelation, spectrum) are set by the parameters of the generators and are the same in every conforming implementation (Cox and Miller 1965; Papoulis and Pillai 2002).

A *realisation* is one waveform drawn from this law. In StimGen 2 a realisation is a deterministic function of three things:

1. the description,
2. the sampling rate f_s ,
3. a *master seed* s , a non-negative integer.

Two conforming implementations given the same three inputs **MUST** produce the same samples, within the reproducibility level of Sec. 11.5.

11.2. Random streams

Every occurrence of a stochastic generator in a waveform is called an *instance*. Each instance draws its random numbers from its own *stream*, an infinite sequence of 64-bit words $w_0, w_1, w_2, \dots$. The stream depends only on a 128-bit *key*.

Rule 5 (generator). The stream for key κ is the output of the counter-based generator Philox4x64-10 (Salmon et al. 2011) with key κ . Block $j = 0, 1, 2, \dots$ of the stream is the output for the 256-bit counter whose four 64-bit words are $(j, 0, 0, 0)$, and its four output words are used in order.

A counter-based generator computes the j -th block of words directly from the key and the counter j , without stepping through the previous ones. Streams with different keys are statistically independent for all practical purposes, and a stream can be read starting at any position.

11.3. Keys and addresses

11.3.1. Addresses

Every instance has an *address* that identifies its position in the description. The address is a text string built from the path that leads from the root of the stimulus to the instance:

- the name of the channel, e.g. `ch=Iinj`;
- for each enclosing sequence, the index of the segment, starting from 0, e.g. `s3`;
- for each enclosing **repeat**, the index of the copy, e.g. `r2`;
- for each enclosing operator or function, the position of the operand, starting from 0, e.g. `o1`.

The components are joined by `/`. For example, the noise in

```
1s  dc(0)
5s  sine(1, 1Hz) + ou(0, 0.2, 5ms)
```

on channel `Iinj` has address `ch=Iinj/s1/o1`. Addresses are taken from the description as written, never from a rewritten form.

11.3.2. Keys

Rule 6 (key derivation). The key of an instance is formed from the first 16 bytes of the SHA-256 digest of a text string (encoded in UTF-8): bytes 0–7 and bytes 8–15, each read as a big-endian unsigned integer, give the two 64-bit key words κ_0 and κ_1 . The string is:

- if the instance has an explicit parameter `seed=n`, the string is `fixed:n`;
- otherwise, the string is `s:a`, where *s* is the master seed written in decimal and *a* is the address.

Two consequences matter in practice.

- **Locality.** The noise of a segment does not depend on what other segments contain. Changing the amplitude of a step in segment 0 does not change the noise realisation in segment 1. (Inserting or removing a segment *before* it does change it, because the addresses shift. Use an explicit **seed** to make a realisation independent of its position.)
- **Frozen noise.** An explicit **seed** gives the same realisation wherever it appears, independently of the master seed and of the address. Two segments with the same primitive, parameters, duration, and seed produce identical samples.

```
200ms ou(2, 0.5, 100ms, seed=21)    # realisation A
50ms  dc(0)
200ms ou(2, 0.5, 100ms, seed=21)    # realisation A again
50ms  dc(0)
200ms ou(2, 0.5, 100ms)             # a different realisation
```

11.3.3. The master seed

The master seed is set by the stimulus or protocol layer, or on the command line. If none is given, the implementation **MUST** draw one from a source of entropy of the operating system, and it **MUST** record the seed it used with the output (Sec. 15). Every realisation that was ever played can therefore be regenerated exactly.

11.4. From words to variates

Generators consume words from their stream in the order defined below. These transformations are part of the specification, because two implementations that transform the same words differently would produce different samples.

- **Uniform** variate on $[0, 1)$: $U = \lfloor w/2^{11} \rfloor \cdot 2^{-53}$, i.e. the top 53 bits of one word.
- **Standard Gaussian** variates are produced in pairs by the Box–Muller transform of two consecutive uniforms U_1, U_2 : $\xi = \sqrt{-2 \ln(1 - U_1)} \cos(2\pi U_2)$ and $\xi' = \sqrt{-2 \ln(1 - U_1)} \sin(2\pi U_2)$, used in this order.
- **Exponential** variate with unit mean: $E = -\ln(1 - U)$.

The consumption order for each stochastic primitive is:

Table 11.1.: Consumption of random variates.

Primitive	Variates consumed, in order
ou	$\xi_0, \xi_1, \dots, \xi_{N_i-1}$, one per sample; ξ_0 is drawn even when init is not stationary
wnoise	$\xi_0, \dots, \xi_{N_i-1}$
unoise	$U_0, \dots, U_{N_i-1}$
pulses (Poisson)	$E_1, E_2, \dots$ for the successive intervals, until an onset exceeds the segment
cnoise	for $j = 1, \dots, \lfloor N_i/2 \rfloor$: the real then the imaginary part of the coefficient at f_j

Drawing ξ_0 in every case keeps sample k of an OU segment tied to variate ξ_k , whichever initial condition is chosen.

11.5. Reproducibility levels

Bit-for-bit equality across implementations is achievable only for operations that IEEE 754 rounds correctly. Elementary functions such as $\sin$, $\exp$, and $\ln$ are not correctly rounded by most mathematical libraries, and their last bit may differ between platforms. We therefore define two levels.

- **Level A (bit-exact)**. The samples are identical bit for bit. This level is required for the words of every stream and for the uniform variates derived from them. It extends to all samples when the implementation uses correctly rounded elementary functions.

- **Level B (numerically equivalent).** Every sample differs from the Level A result by at most a few units in the last place, relative to the amplitude scale of its segment. Conforming implementations **MUST** reach at least Level B, and **MUST** state which level they reach.

Rounding differences do not accumulate in recursive generators: in the OU update Eq. 9.12, an error is multiplied by $\rho < 1$ at every step.

One exception to Level B is possible. A rounding difference can move a Poisson onset across the midpoint between two samples, so that `align=grid` places the pulse one sample earlier or later. Such events are extremely rare. An implementation that claims Level A excludes them.

11.6. Repetitions and trials

Within one waveform, every copy of a **repeat** has its own address and therefore its own realisation (Sec. 10.5).

Across trials, the protocol layer derives the master seed of each trial from a protocol seed and the index of the trial (Sec. 13). Repeated trials thus present different realisations, unless the description fixes the realisation with an explicit **seed** (frozen noise) or the protocol reuses the same master seed for several trials. Both designs are common in experiments, and both are stated explicitly in the description, never left to chance.

12. The stimulus layer

A *stimulus* is what one trial of an experiment plays: several waveforms on named output channels, sampled on a common grid, together with markers that label instants of interest. This chapter defines the stimulus layer. Its text syntax is summarised here and given formally in Sec. 14.

12.1. Definition

Definition 6 (stimulus). A stimulus is a tuple

$$S = (\{(c_j, U_j, r_j, w_j)\}_{j=1}^J, M, f_s^*, s^*),$$

where, for each channel j , c_j is a unique name, U_j a physical unit, r_j a rest value, and w_j a waveform; M is a set of markers; f_s^* is an optional sampling rate; and s^* is an optional master seed.

A stimulus file looks like this:

```
sg 2 stimulus
rate 20kHz
seed 4711

channel Iinj unit=pA rest=0 {
    1s      dc(0)
    @on
    500ms   dc(300)
    1s      dc(0)
}

channel Vcmd unit=mV rest=-70 {
    2.5s    dc(-70)
}

marker sync at 0s, 1s
```

12.2. Channels

A channel is a *logical* output. Its name identifies it in the description, in the output files, and in the addresses of its random streams (Sec. 11.3). A channel has:

- a **unit** U_j , such as pA, nA, mV, nS, or V (required). The samples of the channel are numbers in this unit;
- a **rest value** r_j (default 0), the value the output holds before the first sample, after the last one, and between trials;

- a **waveform** w_j , written inline in braces or taken from a file with use `"name.sg"`.

The stimulus does not say which physical output, D/A converter, or amplifier gain corresponds to a channel. That binding belongs to the configuration of the set-up, outside StimGen 2, so that the same `.sg` file can be played on different rigs. The gain γ used in Eq. 8.6 comes from that binding.

12.2.1. Units and dimensional checks

Each unit has a dimension: current (A with an SI prefix), voltage (V), conductance (S), or dimensionless (1). If an amplitude literal in the waveform of a channel carries a unit (e.g. `dc(0.3nA)` on a channel in `pA`), it **MUST** have the dimension of the channel, and it is converted to the channel unit (here, 300). A literal of another dimension is an error. Literals without a unit are taken in the channel unit.

12.2.2. Copies of a channel

The construct `channel B unit=pA copy A` gives channel B *the same samples* as channel A, noise included. In contrast, two channels that **use** the same waveform file receive different noise realisations, because their stochastic instances have different addresses (Sec. 11.3). Both behaviours are useful, and the description states which one is meant.

12.2.3. Digital channels

A channel declared with `digital NAME { ... }` drives a digital output line, for example a TTL trigger for a camera or a light source. Its waveform is written like any other, but every realised sample **MUST** be exactly 0 or 1; any other value is an error. Digital channels have no unit, and their rest value is 0 unless stated.

12.3. Duration and alignment

All channels share time zero and the sampling grid $t_k = k \Delta t$. Each channel waveform is sampled with the rules of Sec. 8, independently of the others. The duration of the stimulus is the longest channel duration,

$$T_S = \max_j T_{w_j},$$

or the value given by an explicit **duration** statement, which **MUST NOT** be shorter than any channel. A channel that ends earlier holds its rest value until T_S . An implementation **SHOULD** warn when channel durations differ and no explicit **duration** is given, since this is more often an oversight than an intention.

The sampling grid is common to all channels. Different sampling rates for different channels are not supported.

12.4. Markers

A *marker* is a named instant within the stimulus. Markers do not change any output. They are written to the output files with the realised data, so that analysis software can align responses to stimulus events without having to re-derive them (Sec. 15). Markers come from two sources.

1. **Segment labels.** A line `@name` inside a waveform labels the start of the next segment. The marker time is the realised start of that segment, $n_i \Delta t$. Labels must be unique within a channel, and the marker is reported as `channel.name` (e.g. `Iinj.on`).
2. **Explicit markers.** The statement `marker NAME at t1, t2, ...` places markers at the given times, each rounded to the nearest sample with the rounding of Eq. 8.1.

A digital channel can be used to send a marker to external equipment; a marker is only a label in the data.

12.5. Sampling rate and seed

- **rate** states the sampling rate the stimulus requires. If it is present, the implementation MUST use it, or refuse to play the stimulus if the hardware cannot. If it is absent, the rate comes from the configuration of the set-up or from the command line. The rate actually used is always recorded.
- **seed** sets the master seed of the stimulus (Sec. 11.3). A protocol may override it for each trial (Sec. 13.3.3). If no seed is given anywhere, one is drawn and recorded.

12.6. A waveform is also a stimulus

A file or command-line text that contains only a waveform is accepted wherever a stimulus is expected. It is played as a stimulus with one channel, whose name, unit, and rest value come from the configuration (by default: name `out`, the unit configured for the default output, rest value 0). This keeps the simplest case simple.

13. The protocol layer

An experiment rarely plays a single stimulus once. It repeats it, varies a parameter across trials (a family of current steps, a set of frequencies), shuffles the order to avoid slow drifts, and leaves time between trials for the cell to recover. The protocol layer describes these things. It is the only layer with variables, and it is defined entirely by how it *expands* into a list of ordinary stimuli.

13.1. A first protocol

```
sg 2 protocol
seed      2026
sweep     amp = from -300pA to 50pA step 50pA
repeat    2
order     shuffled
period    5s

stimulus {
    channel Iinj unit=pA {
        500ms dc(0)
        @step
        1s    dc($amp)
        500ms dc(0)
    }
}
```

This protocol plays eight current steps, from -300 to 50 pA, twice each, in a random but reproducible order, with one trial every 5 s. Sixteen trials in total.

13.2. Elements of a protocol

Table 13.1.: Statements of a protocol.

Statement	Meaning	Default
<code>stimulus { ... }</code> or <code>stimulus "file.sg"</code>	the stimulus template (required); inline, it may also be a plain waveform	—
<code>sweep name = values</code>	a variable and the values it takes	none
<code>let name = expression</code>	a derived variable	none
<code>repeat n</code>	number of repetitions of every condition	1

Statement	Meaning	Default
<code>order ...</code>	sequential , grouped , shuffled , or shuffled-blocks	sequential
<code>period t</code> or <code>gap t</code>	onset-to-onset time, or pause between trials	gap 0s
<code>seed n</code>	protocol seed	drawn and recorded
<code>noise ...</code>	per-trial , per-condition , or fixed	per-trial
<code>start ...</code>	immediately or on trigger	immediately

13.2.1. Variables and substitution

A variable is defined by `sweep` (it takes a list of values) or by `let` (it is computed from other variables). Inside the stimulus template, `$name` stands for the value of a variable, and `$( expression )` for an arithmetic expression of variables, e.g. `$(amp / 2)`. A substitution may appear wherever a numeric literal may appear: an amplitude, a duration, a frequency, or a **repeat** count. Values carry units. Within `let` and `$( ... )`, quantities may be multiplied and divided, giving derived dimensions such as charge (current $\times$ time), but every substituted value must have the dimension of the field where it appears (Sec. 6.3).

Substitution is textual. Each `$name` or `$( ... )` is replaced by the text of its value before the template is parsed: a value listed in a sweep keeps the form in which it was written (e.g. `-300pA`); a value of a range (`from`, `linspace`, `logspace`) is written in the unit of its first value (e.g. `-300pA`, `-250pA`, ..., `0pA`); a computed value is written in engineering notation with a unit from Table 14.1 (e.g. `400pA`). A computed value whose dimension has no unit there, such as a charge, can be used in other `let` expressions but cannot be substituted.

Variables exist only in protocols. A waveform or stimulus file never contains `$`.

13.2.2. Sweep values

The values of a sweep are written in one of these forms:

- a list: `[10Hz, 20Hz, 40Hz]`;
- an arithmetic range: `from A to B step S`, which includes *B* if it is reached exactly. The values $A + iS$ are computed in exact decimal arithmetic, so that `from -0.3nA to 0.1nA step 0.1nA` gives exactly `-0.3nA`, `-0.2nA`, `-0.1nA`, `0nA`, `0.1nA`;
- `linspace(A, B, n)`: *n* equally spaced values from *A* to *B* inclusive;
- `logspace(A, B, n)`: *n* values equally spaced on a logarithmic scale;
- a list of tuples for variables that change together: `sweep (amp, dur) = [(100pA, 1s), (200pA, 500ms), (400pA, 250ms)]`.

13.2.3. Conditions

With several `sweep` statements, the *conditions* are all combinations of their values (the Cartesian product), enumerated with the first sweep varying slowest. With no sweep there is one condition. The `let` variables are then evaluated for each condition. For example,

```
sweep amp = [100pA, 200pA]
sweep dur = [250ms, 500ms]
let charge = amp * dur
```

defines four conditions: (100, 250), (100, 500), (200, 250), (200, 500).

13.3. Expansion semantics

Definition 7 (expansion). The expansion of a protocol is an ordered list of trials. Each trial consists of:

1. a *trial index* $j = 0, 1, \dots$ in playing order;
2. a *condition index* and a *repetition index*;
3. a fully substituted stimulus, with no variables left;
4. a master seed (Sec. 13.3.3);
5. a start rule (**period**, **gap**, or **trigger**) relative to the previous trial.

Playing a protocol means playing its trials in order, each as an independent stimulus. Expansion is deterministic. The same protocol file and the same protocol seed always give the same list of trials.

13.3.1. Ordering

Let C be the number of conditions and R the number of repetitions.

- **sequential**: for each repetition, all conditions in order. The list is $c_1, c_2, \dots, c_C, c_1, c_2, \dots$, i.e. the whole sweep is run, then run again.
- **grouped**: for each condition, all its repetitions in a row: $c_1, c_1, \dots, c_2, c_2, \dots$
- **shuffled**: a random permutation of all $C \times R$ trials.
- **shuffled-blocks**: R blocks, each one a separate random permutation of the C conditions. Every condition is played once before any is played twice, which balances slow drifts across conditions.

Random permutations are drawn with the Fisher–Yates algorithm, using the stream whose key is derived as in Sec. 11.3 from the string `order:s`, where s is the protocol seed. For a list of length m , for $i = m-1$ down to 1, the element at position i is exchanged with the one at position $\lfloor U(i+1) \rfloor$, where U is the next uniform variate of the stream (Sec. 11.4). For **shuffled-blocks** the R blocks are permuted one after the other, from the same stream. Before shuffling, the list is in **sequential** order.

13.3.2. Timing between trials

- **period t**: each trial starts t after the start of the previous one. If a trial is longer than t , the next one starts as soon as it ends, and the implementation **MUST** record the delay.
- **gap t**: each trial starts t after the end of the previous one.
- **start on trigger**: each trial waits for an external trigger, after the **period** or **gap** has elapsed.

Between trials, every channel holds its rest value.

13.3.3. Seeds of the trials

The master seed of trial j , in condition c and repetition r , is derived from the protocol seed s according to the `noise` statement:

Table 13.2.: Noise policies of a protocol.

noise	Master seed of the trial is derived from	Effect
per-trial	s, j	every trial has a new realisation
per-condition	s, c	repetitions of a condition share one realisation
fixed	s	all trials share one master seed

Trials, conditions and repetitions are numbered from 0. The protocol seed s is the one given by `seed`, unless the program that expands the protocol is given another one (e.g. on the command line); without either, it is drawn and recorded. The derived seed is the first 64 bits of the SHA-256 digest of the string `trial:s:j`, `condition:s:c`, or `fixed:s`, i.e. its first 8 bytes read as a big-endian unsigned integer. Generators with an explicit `seed` are not affected (Sec. 11.3).

`noise per-condition` is the usual design for measuring the reliability of spike timing: the same noise realisation is repeated, and the response variability is attributed to the neuron.

13.4. The directory form

A protocol can also be given as a directory:

```
steps/  
  protocol.sgi  
  0000.sg  
  0001.sg  
  ...
```

Each `.sg` file is one fully substituted stimulus, and the *index* file `protocol.sgi` lists the trials in playing order, one per line, with their start rule and master seed:

```
sg 2 index  
period 5s  
0000.sg seed=13958223740118447071  
0001.sg seed=772146009915234880  
...
```

If the index file is missing, the trials are the `.sg` files of the directory in lexical order of their names, each played once, and the timing comes from the command line.

When it writes a directory form, an implementation SHOULD copy into it the files that the trials refer to by relative names (`use, file(...)`), so that the directory is self-contained.

Equivalence. Every protocol expands into exactly one directory form, and an implementation MUST be able to write it (for example with a command such as `sg expand protocol.sg steps/`). Playing the directory gives the same trials, samples, and seeds as playing the protocol. Conversely, any directory of `.sg` files is a valid protocol. The two forms are two views of the same object.

Design rationale

Why a protocol file, and why also a directory. A directory of stimulus files traversed by a sequencer is a simple and robust way to run a family of stimuli: what is played is exactly what is on disk, and each file can be opened and checked. It has three weaknesses. The family is usually generated by a shell loop, which lives outside the data and is easily lost. Shuffling and its seed are left to the sequencer and are seldom recorded. And relations between parameters (a duration that depends on an amplitude) are hidden in the loop.

A protocol file keeps all of this in one short text that is stored with the data. Defining it by its expansion means that nothing is lost: the directory remains available, as an output of `expand` rather than of a hand-written loop, and a user who prefers to prepare stimuli by other means can still supply a directory. Variables are confined to the protocol layer, so that waveforms stay free of control flow and remain easy to read.

For comparison, the protocol at the beginning of this chapter replaces a loop such as

```
for amp in $(seq -300 50 50); do
    echo "500ms dc(0); 1s dc($amp); 500ms dc(0)" > steps/step_$amp.sg
done
```

followed by a sequencer invocation that must be told separately how many times to repeat, in which order, at which interval, and with which seed.

14. Concrete syntax

This chapter gives the text syntax of `.sg` files: lexical rules, a grammar in Extended Backus–Naur Form (EBNF), and the rules for using the same text on the command line. The meaning of each construct is defined in the previous chapters; here we only fix how it is written.

14.1. Lexical rules

Characters. Files are UTF-8. Outside strings and comments, only ASCII characters are significant; `µs` is accepted as a synonym of `us`.

Comments. A `#` outside a string starts a comment that runs to the end of the line.

Whitespace and line breaks. Spaces and tabs separate tokens and are otherwise ignored. A line break *separates segments* (like `;`) when it occurs directly inside a sequence, i.e. at the top level of a waveform or directly inside braces. A line break is ignored, and the expression continues on the next line, when it occurs

- inside round parentheses or square brackets, or
- after a binary operator (`+` `-` `*` `/`), a comma, or `=`.

Blank lines and lines holding only a comment are ignored.

Identifiers. `[A-Za-z_][A-Za-z0-9_]*`. Names are case-sensitive. Keywords (such as `repeat`, `prev`, `channel`, `sweep`) are recognised only in the positions where the grammar expects them, so that, for instance, `from` and `to` remain usable as parameter names of `ramp`.

Numbers. `[0-9]+(\.[0-9]*)?([eE][+-]?[0-9]+)?` or a leading-dot form `\.[0-9]+(\. ...)`. A sign is an operator, not part of the number.

Quantities. A number followed *immediately* (without a space) by a unit is a quantity, e.g. `500ms`, `20kHz`, `-300pA` (unary minus applied to `300pA`). The units are:

Table 14.1.: Unit suffixes. Units are case-sensitive: `ms` is a millisecond, `mS` a millisiemens.

Dimension	Units
time	s, ms, us (μ s), ns, min
frequency	Hz, kHz, MHz
phase	rad, deg
fraction	%
current	fA, pA, nA, uA, mA, A
voltage	uV, mV, V
conductance	pS, nS, uS, mS, S

Strings. Text in double quotes, with `\`" and `\\` as the only escapes. Strings are used for file names.

14.2. Grammar

The grammar below uses ISO EBNF conventions: { *x* } is zero or more repetitions, [*x*] is optional, | separates alternatives, and quoted text is literal. SEP stands for ; or a separating line break, as defined above.

14.2.1. Files

```
file      = [ header ] ( waveform | stimulus | protocol | index ) ;
header    = "sg" , version , kind , SEP ;
version   = number ;                               (* "2" for this specification *)
kind      = "waveform" | "stimulus" | "protocol" | "index" ;
```

A file without a header is a waveform. The *kind* in the header MUST match the content.

14.2.2. Waveforms

```
waveform  = sequence ;
sequence  = item , { SEP , item } ;
item      = label | segment ;
label     = "@" , identifier ;
segment   = [ label ] , [ duration ] , expr ;
duration  = quantity | substitution ;              (* of dimension time *)

expr      = term , { ( "+" | "-" ) , term } ;
term      = unary , { ( "*" | "/" ) , unary } ;
unary     = [ "-" ] , primary ;
primary   = number | quantity | "prev" | substitution
           | call | block | repeat | "(" , expr , ")" ;

call      = identifier , "(" , [ args ] , ")" ;
args      = arg , { "," , arg } ;
arg       = [ identifier , "=" ] , value ;
value     = expr | string | identifier | list ;
list      = "[" , [ value , { "," , value } ] , "]" ;

block     = "{" , sequence , "}" ;
repeat    = "repeat" , count , block ;
count     = number | substitution ;                (* a non-negative integer *)
```

call covers primitives (Sec. 9), maps (Table 10.3), and the binary functions min and max. An identifier as a value is a keyword argument such as law=exp or clock=global.

Duration rule. A segment that begins with a quantity of dimension time takes that quantity as its duration. Since amplitudes never have the dimension of time, this resolves the only potential ambiguity: 1s -0.5 is a segment of 1 s with value -0.5, not the difference 1s - 0.5.

A label on its own line labels the next segment; a label may also precede a segment on the same line, as in @on 500ms dc(300).

14.2.3. Stimuli

```
stimulus      = stim_stmt , { SEP , stim_stmt } ;
stim_stmt     = "rate" , quantity
               | "seed" , ( number | substitution )
               | "duration" , quantity
               | "channel" , identifier , { attribute } , source
               | "digital" , identifier , { attribute } , source
               | "marker" , identifier , "at" , quantity , { "," , quantity } ;
attribute     = ( "unit" , "=" , unit ) | ( "rest" , "=" , expr ) ;
source        = block | ( "use" , string ) | ( "copy" , identifier ) ;
```

14.2.4. Protocols

```
protocol      = prot_stmt , { SEP , prot_stmt } ;
prot_stmt     = "stimulus" , ( "{" , stimulus , "}" | string )
               | "sweep" , names , "=" , values
               | "let" , identifier , "=" , expr
               | "repeat" , number
               | "order" , ( "sequential" | "grouped" | "shuffled" | "shuffled-blocks" )
               | ( "period" | "gap" ) , quantity
               | "seed" , number
               | "noise" , ( "per-trial" | "per-condition" | "fixed" )
               | "start" , ( "immediately" | "on" , "trigger" ) ;
names         = identifier | "(" , identifier , { "," , identifier } , ")" ;
values        = list
               | "from" , expr , "to" , expr , "step" , expr
               | ( "linspace" | "logspace" ) ,
                 "(" , expr , "," , expr , "," , number , ")" ;
substitution  = "$" , identifier | "$(" , expr , ")" ;
```

Inside the expressions of `let` and `$( ... )`, identifiers denote variables. Substitutions are allowed only in the stimulus template of a protocol. They are replaced textually before the template is parsed as a stimulus (Sec. 13), so the expanded stimulus contains no `$`.

14.2.5. Index files

```
index         = { index_stmt , SEP } ;
index_stmt    = ( "period" | "gap" ) , quantity
               | "start" , ( "immediately" | "on" , "trigger" )
               | string_or_name , [ "seed" , "=" , number ] ;
```

where `string_or_name` is a file name, quoted if it contains spaces.

14.3. Positional and named arguments

The parameters of a call can be given by position, in the order of the parameter table of the primitive, by name, or both, with all positional arguments first. Each parameter may be given at most once. Thus `sine(3, 1Hz)`, `sine(amp=3, freq=1Hz)`, and `sine(3, freq=1Hz)` are

equivalent. We recommend named arguments for every parameter after the first two, which keeps descriptions readable months later.

14.4. Canonical form

Every description has a *canonical form*, used in provenance records and for comparing descriptions (Sec. 15). The canonical form is obtained by: removing comments and blank lines; writing one segment or statement per line with single spaces between tokens; writing all arguments by name, in the order of the parameter table, including defaults; and writing every quantity in its canonical unit with the shortest decimal representation (e.g. 5ms becomes 0.005s). Two descriptions with the same canonical form have the same meaning.

14.5. Command line

The same text can be passed as a command-line argument instead of a file. Since line breaks are inconvenient there, segments are separated by `;`, and the text is quoted to protect it from the shell:

```
sg render --rate 20kHz '2s dc(0); 500ms dc(300); 2s dc(0)'
```

Because a segment is a single argument, negative values need no special treatment, unlike programs whose options begin with a dash.

The names and options of the reference tools are not part of this specification. As an illustration, a minimal tool set could offer:

Table 14.2.: An illustrative command-line tool set (informative).

Command	Purpose
<code>sg check FILE</code>	parse and run the static checks of Sec. 10.7
<code>sg canon FILE</code>	print the canonical form
<code>sg render FILE</code>	realise samples to an output file (Sec. 15)
<code>sg render PROTOCOL</code>	realise every trial, one output file per trial
<code>sg render DIR</code>	realise the trials of a directory form
<code>sg expand PROTOCOL DIR</code>	write the directory form of a protocol (Sec. 13.4)
<code>sg play FILE</code>	play a stimulus or protocol on the configured hardware

15. Output and provenance

A description is short, but the samples it produces are what reach the preparation. This chapter specifies how realised samples are stored, and which information **MUST** accompany them so that every stimulus that was ever played can be identified and regenerated exactly.

15.1. Realised sample files

An implementation that writes realised samples to a file for exchange **MUST** support the format described here, with the extension `.sgb` (StimGen binary). It is deliberately simple and portable.

Table 15.1.: Layout of a `.sgb` file.

Offset	Size (bytes)	Content
0	8	magic bytes SGB followed by five zero bytes
8	8	length L of the header, unsigned 64-bit integer, little-endian
16	L	header: a JSON object, UTF-8, padded with spaces so that L is a multiple of 8
$16 + L$	$8 \times J \times N$	samples: IEEE 754 binary64, little-endian, channel after channel

The JSON header contains at least:

- **"version"**: the version of this specification (`"2"`);
- **"rate"**: the sampling rate in Hz;
- **"samples"**: the number of samples per channel, N ;
- **"channels"**: a list of objects with **"name"**, **"unit"**, **"rest"**, and **"digital"** (true or false), in the order of the sample blocks;
- **"markers"**: a list of objects with **"name"** and **"sample"** (the sample index);
- **"provenance"**: the provenance record (Sec. 15.2).

Because L is a multiple of 8, the samples start on an 8-byte boundary, so that a reader can map them directly into memory as an array of doubles (several languages require this alignment). Trailing spaces are allowed by JSON, so a reader that parses the whole header is not affected.

All channels have the same number of samples. The samples of channel 1 come first (N values), then those of channel 2, and so on. All multi-byte values are little-endian, and all sizes are 64-bit, so a file written on one machine is read identically on any other.

Note

Recording software usually stores data in its own container, often HDF5. There, the realised stimulus SHOULD be stored as one dataset per channel, with the fields of the JSON header (and in particular the provenance record) as attributes. The `.sgb` format is meant for exchange and for testing implementations against each other.

15.2. The provenance record

Every realisation, whether written to a file, played on hardware, or both, MUST produce a provenance record with the following fields.

Table 15.2.: Fields of the provenance record.

Field	Content
<code>spec_version</code>	version of this specification
<code>implementation</code>	name and version of the implementation
<code>repro_level</code>	A or B (Sec. 11.5)
<code>description</code>	the full text of the description as given, with every file it uses (<code>use, stimulus "..."</code>) included verbatim
<code>canonical_sha256</code>	SHA-256 digest of the canonical form (Sec. 14)
<code>files</code>	for each <code>file(...)</code> primitive: path and SHA-256 digest of the data file
<code>rate</code>	sampling rate actually used (Hz)
<code>master_seed</code>	master seed actually used, whether given or drawn
<code>trial</code>	for protocols: trial, condition, and repetition indices, the substituted variable values, the protocol seed, order, noise policy, and timing
<code>protocol</code>	for protocols: the full text of the protocol (or of the index file)
<code>samples_sha256</code>	SHA-256 digest of the sample blocks, as laid out in Table 15.1
<code>warnings</code>	warnings issued, e.g. empty segments or frequencies above Nyquist
<code>clipping</code>	number and positions of clipped samples (Sec. 8.8), if known
<code>timestamp</code>	date and time of realisation (ISO 8601, UTC)

The record is a JSON object. A record for a protocol trial is a record for the stimulus of that trial, whose `description` is the fully substituted stimulus text, with the fields `trial` and `protocol` added. A single trial can therefore be regenerated from its own record, without the protocol.

Regeneration. Given the fields `description`, `rate`, and `master_seed`, any conforming implementation regenerates the realisation. The digest `samples_sha256` lets one verify the result: equality is guaranteed at Level A, and at Level B the samples agree within the tolerance of Sec. 11.5.

Design rationale

Recording the description and the seed costs a few hundred bytes per trial and makes the stimulus of every recorded sweep recoverable years later, even if the original files are lost. The digest of the samples protects against the opposite problem, an implementation that changes its output between versions without anyone noticing.

15.3. Errors and warnings

Errors stop realisation, and no samples are played or written. They are: syntax errors, failed static checks (Sec. 10.7), non-finite samples (Rule 4), invalid values on digital channels, unreadable files, and a required sampling rate that the hardware cannot provide.

Warnings do not stop realisation, and they are recorded in the provenance. They include: a segment of positive duration with no sample; a frequency above $f_s/2$; a pulse narrower than Δt ; channels of different durations without an explicit `duration`; a trial that could not start at its `period`; and clipped samples.

A. Glossary

- Address** The position of a stochastic generator instance in a stimulus, written as a path such as `ch=Iinj/s1/o1`. It determines the random stream of the instance unless a seed is given (Sec. 11.3).
- Block** A sequence of segments in braces, `{ ... }`. A block is a timed waveform and can be used as an operand (Sec. 10.1).
- Canonical form** A normalised text of a description, with all arguments named and all quantities in canonical units. Two descriptions with the same canonical form have the same meaning (Sec. 14).
- Channel** A named logical output of a stimulus, with a unit, a rest value, and a waveform (Sec. 12.2).
- Concatenation** Joining timed waveforms one after the other in time, written with `;` or a line break (Sec. 7.4).
- Condition** One combination of the values of the sweep variables of a protocol (Sec. 13).
- Digital channel** A channel whose samples must be 0 or 1, used for trigger and TTL outputs (Sec. 12.2).
- Directory form** A protocol given as a directory of fully substituted `.sg` files and an index file. It is equivalent to the protocol it was expanded from (Sec. 13.4).
- Duration-aware** A generator whose values depend on the duration of its segment, such as `ramp`, `chirp`, or `cnoise` (Sec. 7.2).
- End value** The value a segment would take at the first sample after it, for a deterministic segment, or its last sample, for a stochastic one. The keyword `prev` refers to it (Sec. 8.4, Sec. 10.6).
- Expansion** The deterministic conversion of a protocol into an ordered list of trials (Sec. 13.3).
- Frozen noise** A noise realisation that is repeated identically, obtained with an explicit `seed` or with the protocol policy `noise per-condition` (Sec. 11.3, Sec. 13.3.3).
- Generator** A rule that gives a value for every instant of a segment once its duration is known. A generator has no duration of its own (Sec. 7.2).
- Grid-aligned local time** The local time at which a sample is evaluated, $u_k = (k - n_i)\Delta t$, so that every segment starts exactly on a sample (Sec. 8.3).
- Header** The first line of a `.sg` file, e.g. `sg 2 stimulus`, stating the version and the kind of description (Sec. 14).
- Index file** The file `protocol.sgi` of a directory form, listing the trials in playing order with their seeds and timing (Sec. 13.4).
- Label** A name written `@name` before a segment. It produces a marker at the start of that segment (Sec. 12.4).
- Local time** Time measured from the start of the current segment, u .
- Marker** A named instant in a stimulus, stored with the data to help analysis (Sec. 12.4).
- Master seed** The integer from which the random streams of a stimulus are derived (Sec. 11.3).
- Primitive** One of the built-in generators of Sec. 9, such as `sine` or `ou`.
- Protocol** A description of a set of trials: a stimulus template, sweeps, repetitions, ordering, and timing (Sec. 13).
- Provenance record** The information stored with every realisation that allows it to be identified and regenerated (Sec. 15.2).
- Quantity** A number with a unit suffix, such as `500ms` or `-300pA` (Sec. 14.1).

Realisation The definite sequence of samples obtained from a description for a given sampling rate and master seed (Sec. 7.7).

Realised duration The duration of a segment after sampling, $\hat{T}_i = N_i \Delta t$ (Sec. 8.3).

Rest value The value a channel holds before and after its waveform and between trials (Sec. 12.2).

Segment A timed waveform that is one element of a sequence; usually written as a duration followed by an expression (Sec. 7.3).

Sequence Segments joined by concatenation (Sec. 7.4).

Stimulus Waveforms on named channels with a common time base, plus markers: what one trial plays (Sec. 12).

Stream The sequence of random 64-bit words used by one stochastic generator instance (Sec. 11).

Sweep A protocol variable together with the list of values it takes (Sec. 13).

Timed waveform A function of time on an interval of known duration (Sec. 7.3).

Trial One playing of a stimulus within a protocol (Sec. 13.3).

Unary map A function applied to every sample of a waveform, such as `abs` or `clip` (Sec. 10.4).

Waveform The description of the values of one channel during one trial: a sequence of segments (Sec. 7.1).

B. Mathematical derivations

This appendix collects the derivations behind formulas stated in the reference chapters.

B.1. Error of segment boundaries

Rounding to the nearest integer satisfies $|\mathbf{R}(x) - x| \leq \frac{1}{2}$. With $n_i = \mathbf{R}(\tau_i f_s)$ (Eq. 8.1), the realised start time of every segment satisfies

$$|n_i \Delta t - \tau_i| \leq \frac{1}{2} \Delta t,$$

independently of i , so the error does not grow along the sequence. The number of samples of a segment, $N_i = n_{i+1} - n_i$, satisfies $|N_i - T_i f_s| \leq 1$. Equality can occur only through the tie-breaking rule; for example, at $f_s = 1$ Hz, a segment of 1 s starting at 0.5 s spans the boundaries $\mathbf{R}(0.5) = 0$ and $\mathbf{R}(1.5) = 2$, i.e. 2 samples. Rounding each segment separately would instead give a cumulative error of up to $m \Delta t/2$ after m segments.

B.2. Exact update of the Ornstein–Uhlenbeck process

The solution of Eq. 9.11 over an interval of length Δt is

$$X(u + \Delta t) = \mu + (X(u) - \mu) e^{-\Delta t/\tau} + \sigma \sqrt{\frac{2}{\tau}} \int_0^{\Delta t} e^{-(\Delta t-s)/\tau} dW(u + s).$$

The stochastic integral is Gaussian with zero mean and, by the Itô isometry, variance

$$\frac{2\sigma^2}{\tau} \int_0^{\Delta t} e^{-2(\Delta t-s)/\tau} ds = \sigma^2(1 - e^{-2\Delta t/\tau}) = \sigma^2(1 - \rho^2),$$

with $\rho = e^{-\Delta t/\tau}$. It is independent of $X(u)$ and of the integrals over other intervals. This gives Eq. 9.12, which is exact for any Δt (Gillespie 1996). If x_0 is drawn from the stationary law $\mathcal{N}(\mu, \sigma^2)$, then every x_k has the same law, and $\text{Cov}(x_k, x_{k+l}) = \sigma^2 \rho^l = \sigma^2 e^{-l\Delta t/\tau}$, which is the autocorrelation of the continuous process at lag $l \Delta t$.

For comparison, the Euler–Maruyama scheme $x_{k+1} = x_k - (x_k - \mu)\Delta t/\tau + \sigma\sqrt{2\Delta t/\tau}\xi_{k+1}$ has stationary variance $\sigma^2/(1 - \Delta t/(2\tau))$, which is wrong by about $\Delta t/(2\tau)$ in relative terms and diverges as $\Delta t \rightarrow 2\tau$.

The power spectral density follows from the Wiener–Khinchin theorem (Papoulis and Pillai 2002). The two-sided spectrum of $\sigma^2 e^{-|s|/\tau}$ is $2\sigma^2\tau/(1 + (2\pi f\tau)^2)$, and the one-sided spectrum is twice that.

B.3. Peak and normalisation of the biexponential

For $v \geq 0$ let $h(v) = e^{-v/\tau_d} - e^{-v/\tau_r}$ with $\tau_r < \tau_d$. Setting $h'(v) = 0$ gives $e^{-v/\tau_d}/\tau_d = e^{-v/\tau_r}/\tau_r$, hence

$$v^* = \frac{\tau_r \tau_d}{\tau_d - \tau_r} \ln \frac{\tau_d}{\tau_r}.$$

Substituting, $e^{-v^*/\tau_d} = (\tau_r/\tau_d)^{\tau_r/(\tau_d - \tau_r)}$ and $e^{-v^*/\tau_r} = (\tau_r/\tau_d)^{\tau_d/(\tau_d - \tau_r)}$, so the peak value $K = h(v^*)$ is Eq. 9.8. Dividing by K makes the peak equal to 1, and multiplying by `amp` makes it equal to `amp`.

Limit $\tau_r \rightarrow \tau_d$. Put $\tau_r = \tau$ and $\tau_d = \tau + \varepsilon$. To first order in ε , $h(v) \approx \varepsilon (v/\tau^2) e^{-v/\tau}$, whose peak is at $v = \tau$ with value $\varepsilon/(e\tau)$. The normalised waveform tends to $(v/\tau) e^{1-v/\tau}$, the alpha function Eq. 9.9.

B.4. Phase of the chirp

The phase is the integral of the instantaneous angular frequency, $\Phi(u) = 2\pi \int_0^u f(s) ds$.

- Linear law, $f(s) = f_0 + (f_1 - f_0) s/\hat{T}$: $\Phi(u) = 2\pi(f_0 u + (f_1 - f_0)u^2/(2\hat{T}))$, which is Eq. 9.5.
- Exponential law, $f(s) = f_0 k^{s/\hat{T}}$ with $k = f_1/f_0$: $\Phi(u) = 2\pi f_0 \hat{T} (k^{u/\hat{T}} - 1)/\ln k$, which is Eq. 9.6. As $k \rightarrow 1$, $(k^x - 1)/\ln k \rightarrow x$, and the phase tends to $2\pi f_0 u$.

The time the exponential chirp spends between frequencies f and $2f$ is $\hat{T} \ln 2 / \ln k$, the same for every octave.

B.5. Spectrum of discrete white noise

Independent samples with variance σ^2 taken every Δt have a flat two-sided spectral density $\sigma^2 \Delta t$ on $[-f_s/2, f_s/2]$, since its integral over that band must equal σ^2 . The one-sided density on $[0, f_s/2]$ is $2\sigma^2 \Delta t$. Hence a fixed spectral density S_0 (one-sided) requires $\sigma = \sqrt{S_0 f_s/2}$, as stated in Sec. 9.6.2.

B.6. Synthesis of power-law noise

The procedure for `cnoise` follows Timmer and König (1995). Let $M = N_i$ be the number of samples of the segment, Δt the sampling interval, and $f_j = j/(M\Delta t)$ for $j = 0, \dots, \lfloor M/2 \rfloor$.

1. **Target spectrum.** $S_j = f_j^{-\alpha}$ if `fmin` $\leq f_j \leq$ `fmax` and $j \geq 1$; otherwise $S_j = 0$. In particular $S_0 = 0$, so the mean is set only by `mean`.
2. **Random coefficients.** For $j = 1, \dots, \lfloor M/2 \rfloor$, draw a_j and then b_j , standard Gaussian (Table 11.1), and set $Z_j = \sqrt{S_j/2} (a_j + i b_j)$. If M is even, the coefficient at the Nyquist frequency is real, $Z_{M/2} = \sqrt{S_{M/2}} a_{M/2}$, and $b_{M/2}$ is drawn but not used. Set $Z_0 = 0$.
3. **Hermitian symmetry.** $Z_{M-j} = \overline{Z_j}$ for $1 \leq j < M/2$, so that the result is real.
4. **Inverse transform.** $y_k = \sum_{j=0}^{M-1} Z_j e^{2\pi i j k/M}$, for $k = 0, \dots, M-1$.

5. **Scaling.** The expected variance of y_k is

$$V = \sum_{1 \leq j < M/2} 2S_j + \mathbb{1}[M \text{ even}] S_{M/2},$$

since each conjugate pair contributes $2 \operatorname{Re}(Z_j e^{i\phi})$ with variance $2S_j$, and the Nyquist term contributes $S_{M/2}$. The samples are $x_k = \mu + \sigma y_k / \sqrt{V}$. If $V = 0$ (no frequency inside the band), realisation stops with an error.

Because the realisation is built from the whole segment at once, it is periodic with period $M\Delta t$: the last sample connects smoothly to the first. This is a known property of spectral synthesis, and it is the price of an exact spectrum.

References

- Bradner, Scott. 1997. *Key Words for Use in RFCs to Indicate Requirement Levels*. RFC 2119, Internet Engineering Task Force. <https://doi.org/10.17487/RFC2119>.
- Cox, David R., and Hilton D. Miller. 1965. *The Theory of Stochastic Processes*. Chapman; Hall.
- Gillespie, Daniel T. 1996. “Exact Numerical Simulation of the Ornstein-Uhlenbeck Process and Its Integral.” *Physical Review E* 54 (2): 2084–91. <https://doi.org/10.1103/PhysRevE.54.2084>.
- Linaro, Daniele, João Couto, and Michele Giugliano. 2014. “Command-Line Cellular Electrophysiology for Conventional and Real-Time Closed-Loop Experiments.” *Journal of Neuroscience Methods* 230: 5–19. <https://doi.org/10.1016/j.jneumeth.2014.04.003>.
- Papoulis, Athanasios, and S. Unnikrishna Pillai. 2002. *Probability, Random Variables, and Stochastic Processes*. 4th ed. McGraw-Hill.
- Puil, E., B. Gimbarzevsky, and R. M. Miura. 1986. “Quantification of Membrane Properties of Trigeminal Root Ganglion Neurons in Guinea Pigs.” *Journal of Neurophysiology* 55 (5): 995–1016. <https://doi.org/10.1152/jn.1986.55.5.995>.
- Salmon, John K., Mark A. Moraes, Ron O. Dror, and David E. Shaw. 2011. “Parallel Random Numbers: As Easy as 1, 2, 3.” *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 1–12. <https://doi.org/10.1145/2063384.2063405>.
- Timmer, J., and M. König. 1995. “On Generating Power Law Noise.” *Astronomy and Astrophysics* 300: 707–10.
- Uhlenbeck, George E., and Leonard S. Ornstein. 1930. “On the Theory of the Brownian Motion.” *Physical Review* 36 (5): 823–41. <https://doi.org/10.1103/PhysRev.36.823>.